Cyprus University of Technology

CEI 227 Προχωρημένη Αντικειμενοστρεφής Μεθοδολογία και Προγραμματισμός (UML, Java)

Διαδικασία Ανάπτυξης Λογισμικού

Παναγιώτης Ηλία

Διαδικασία Ανάπτυξης Λογισμικού

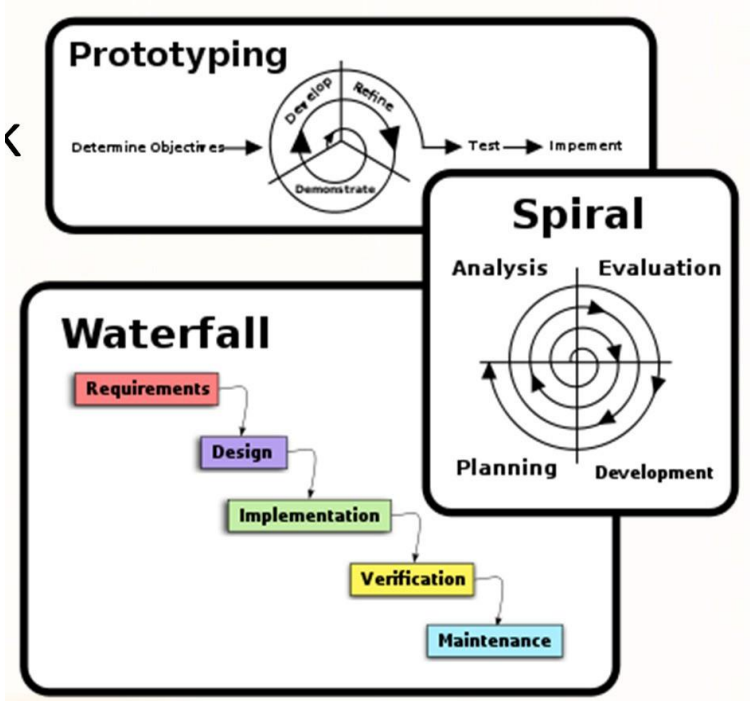
- **Διαδικασία Ανάπτυξης Λογισμικού (software process)**: Ένα δομημένο σύνολο από ενέργειες που χρειάζονται για να αναπτυχθεί ένα λογισμικό
  - Υπάρχουν πολλές διαδικασίες ανάπτυξης λογισμικού (software models/frameworks) με πολλές ιδιαιτερότητες
- **Μοντέλο Διαδικασίας Ανάπτυξης Λογισμικού (software process model)**: Είναι μία αφηρημένη/απλοποιημένη αναπαράσταση της διαδικασίας ανάπτυξης λογισμικού
  - Κάθε στάδιο της διαδικασίας ανάπτυξης λογισμικού αναγνωρίζεται:
  - Το μοντέλο (process model) αναπαριστά τις δραστηριότητες (activities) που σχετίζονται άμεσα με το κάθε στάδιο
  - Η συλλογή των μοντέλων αναπαριστά την διαδικασία ανάπτυξης λογισμικού

Δραστηριότητες των Μοντέλων Διαδικασίας Ανάπτυξης Λογισμικού

- Βασικές Δραστηριότητες των μοντέλων συμπεριλαμβάνουν:
  - **Απαιτήσεις**: Συλλογή απαιτήσεων σε διάφορα επίπεδα (επίπεδο οργανισμού, στρατηγικό επίπεδο)
  - **Ανάλυση**: Κατανόηση της φύσης του λογισμικού που θα αναπτυχθεί, το πεδίο πληροφόρησης, απαραίτητες λειτουργίες, συμπεριφορά, απόδοση και interfaces
  - **Σχεδίαση**: Αναγνώριση/Σχεδίαση δομών δεδομένων, αλγόριθμων, αρχιτεκτονικής προγράμματος και interfaces
  - **Υλοποίηση**: μετάφραση των απαιτήσεων και της ανάλυσης σε εκτελέσιμο κώδικα
  - **Συντήρηση**: τροποποίηση του λογισμικού μετά την παράδοση του στον πελάτη

Γενικά Μοντέλα Διαδικασίας Ανάπτυξης Λογισμικού

- **Waterfall**: Γραμμικό framework
- **Prototyping**: Επαναληπτικό framework
- **Αυξητικό**: Συνδυασμός γραμμικού/επαναληπτικού
- **Spiral**: Συνδυασμός γραμμικού/επαναληπτικού framework, έμφαση στην εκτίμηση του κινδύνου (risk assessment)
- **Rapid Application Development (RAD):** Επαναληπτικό, έμφαση στη γρήγορη ανάπτυξη
- **Extreme Programming**: καινούριες εκδόσεις δημιουργούνται πολύ συχνά, μικροί κύκλοι ανάπτυξης, σχεδιασμένο για συχνές αλλαγές απαιτήσεων

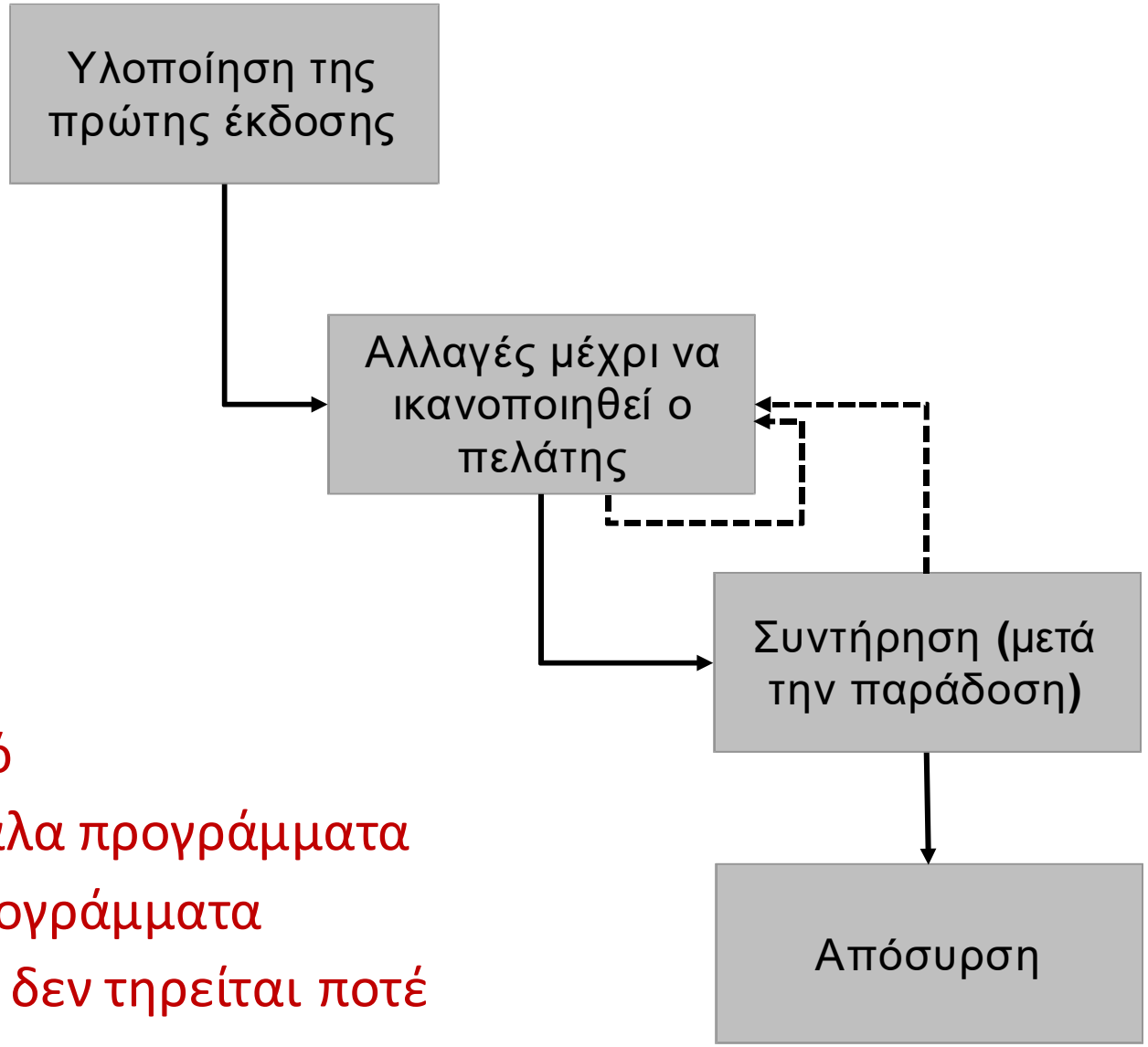


Build & Fix

- Καθόλου απαιτήσεις!
- Καθόλου Σχεδίαση!
- Καθόλου Προδιαγραφές!

+ Πολύ αποδοτικό οικονομικά για μικρά προγράμματα

- Χαμηλής ποιότητας λογισμικό
- Καθόλου αποδοτικό για μεγάλα προγράμματα
- Υψηλό κόστος για μεγάλα προγράμματα
- Ο χρόνος παράδοσης σχεδόν δεν τηρείται ποτέ
- Εφιάλτης Συντήρησης

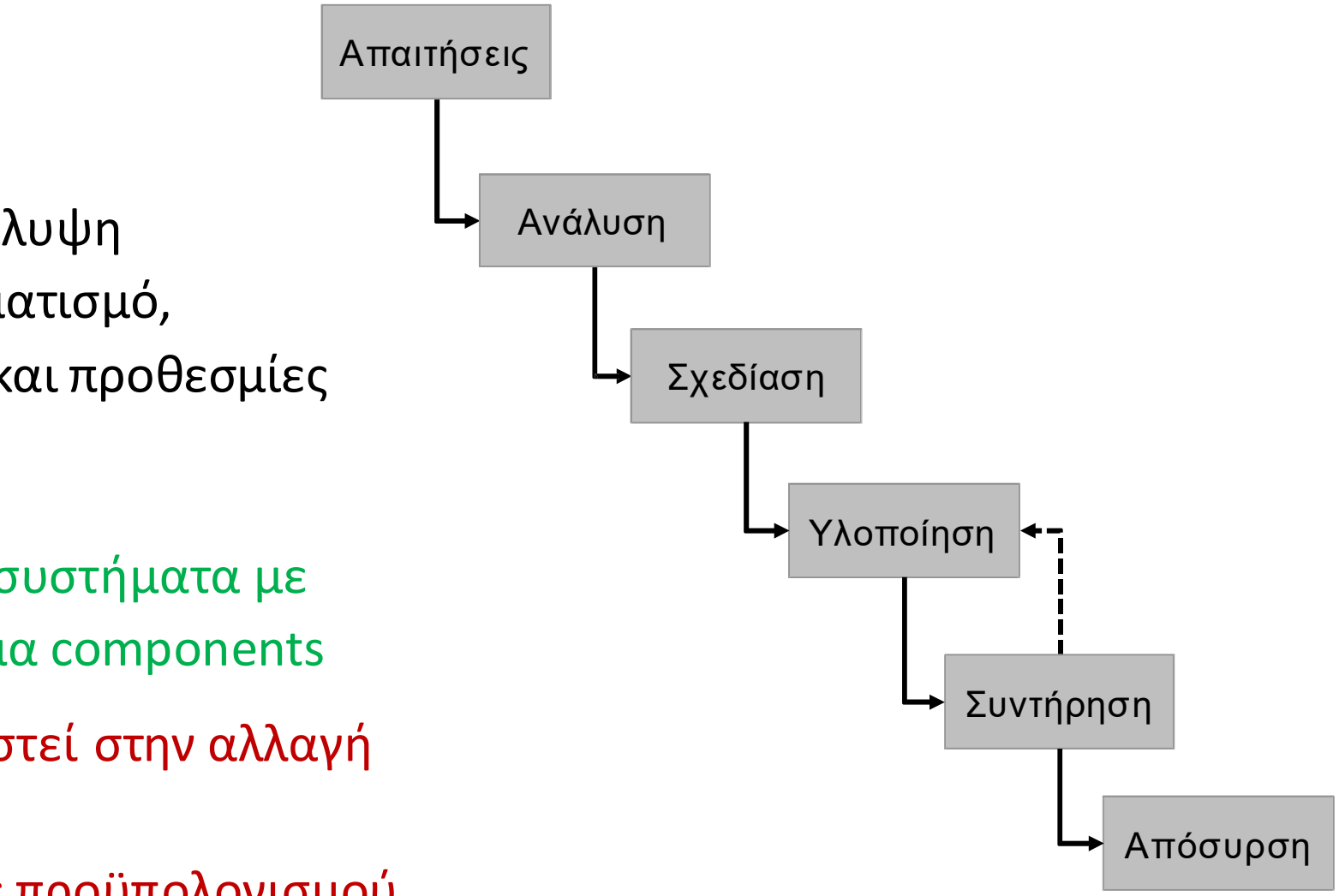


Waterfall Model

- Γραμμικό μοντέλο
- Διαδοχικές φάσεις με
  κάποια μικρή επικάλυψη
- Έμφαση στον προγραμματισμό,
  προϋπολογισμούς και προθεσμίες

+ Διακριτές φάσεις
+ Αποδοτικό για μεγάλα συστήματα με
  ξεχωριστά αυτόνομα components

- Δύσκολο να προσαρμοστεί στην αλλαγή
- Όχι πολύ ευέλικτο
- Συνήθως βγαίνεις εκτός προϋπολογισμού

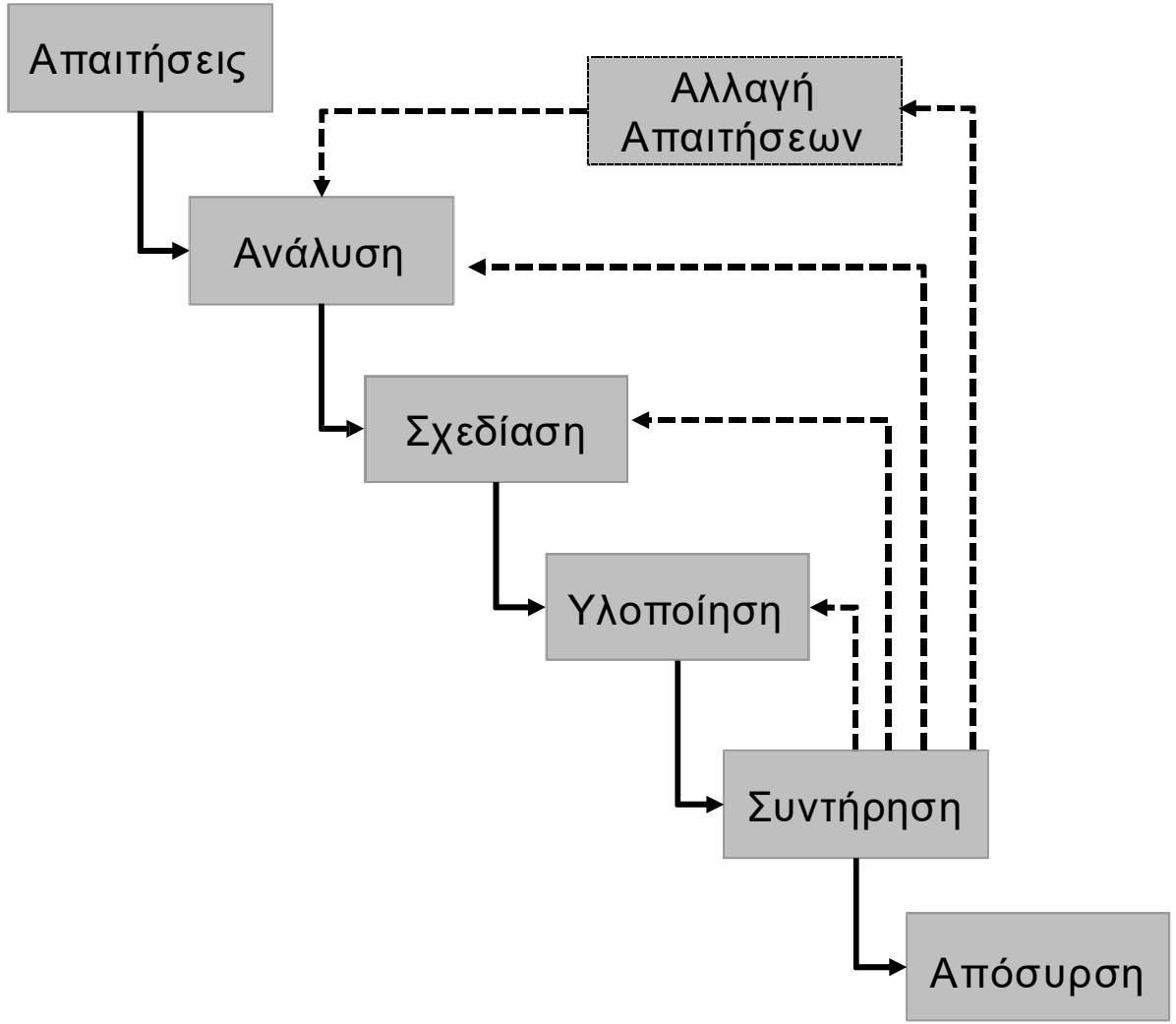


Waterfall Model + feedback loops

Ανατροφοδότηση από τη συντήρηση σε προηγούμενες φάσεις

+ Πιο ευέλικτο από το απλό waterfall model

- Όχι κατάλληλο για μικρά προγράμματα

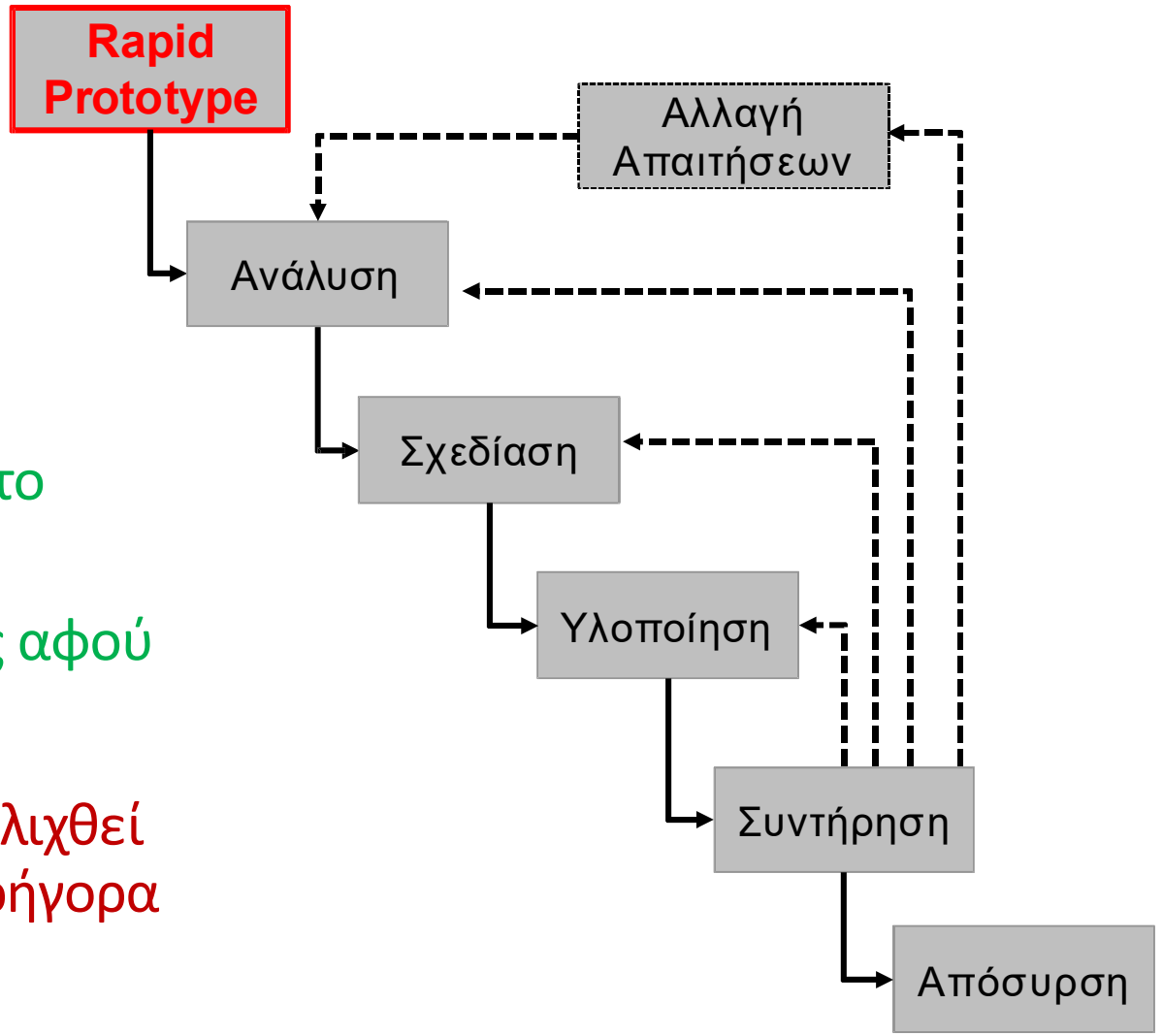


Rapid Prototyping

- Βασισμένο σε Πρωτότυπα: ατελείς εκδόσεις του λογισμικού
- Διατίθενται από τα αρχικά στάδια του προγράμματος

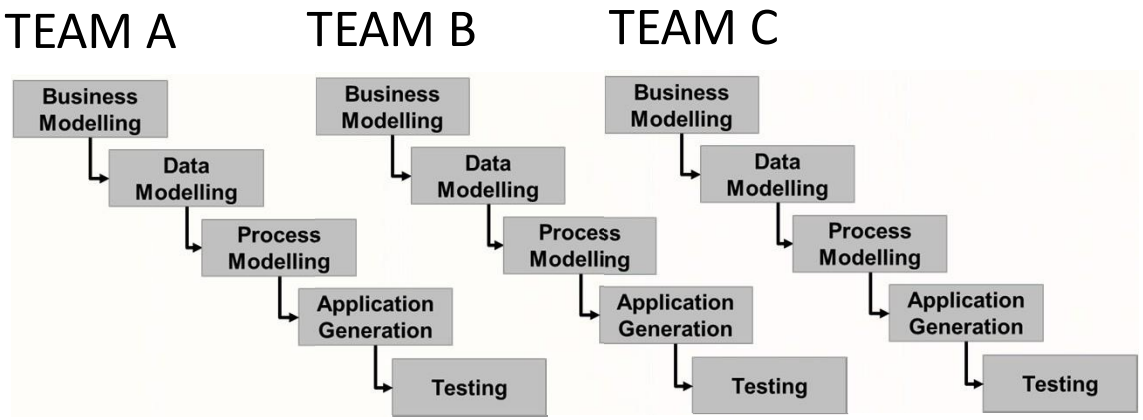
+ Οι χρήστες πειραματίζονται γρήγορα με το σύστημα → πιο ποιοτικές απαιτήσεις
+ Πιο αποδεκτό από τους τελικούς χρήστες αφού έχουν εμπλακεί στην υλοποίηση

- Συνήθως γίνεται συμβιβασμός για να εξελιχθεί το πρωτότυπο ως το τελικό προϊόν πιο γρήγορα
- Χρειάζεται αρκετή εμπειρία από τους προγραμματιστές

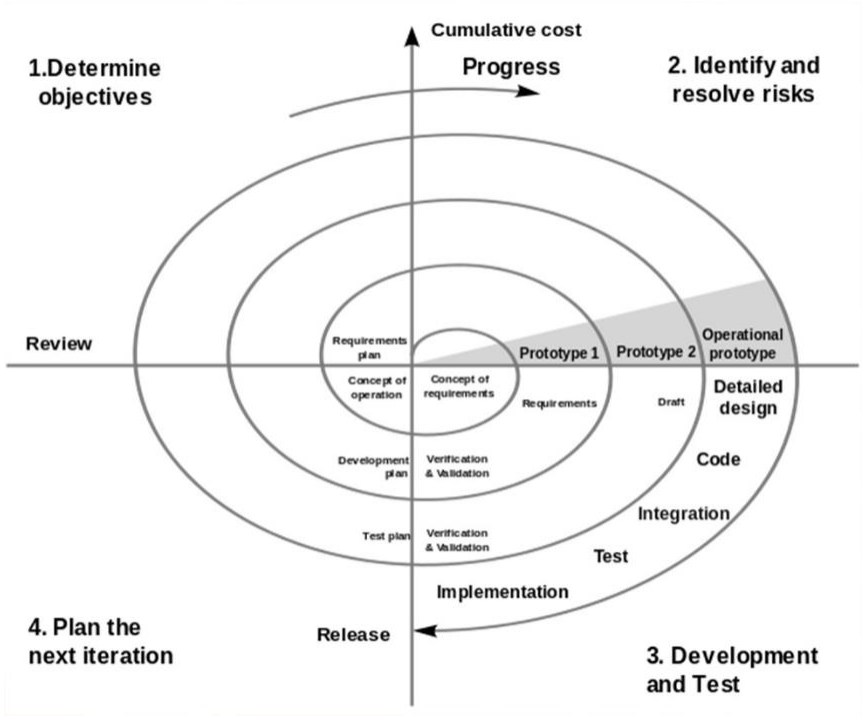


Άλλα γνωστά μοντέλα

Rapid Application Development RAD



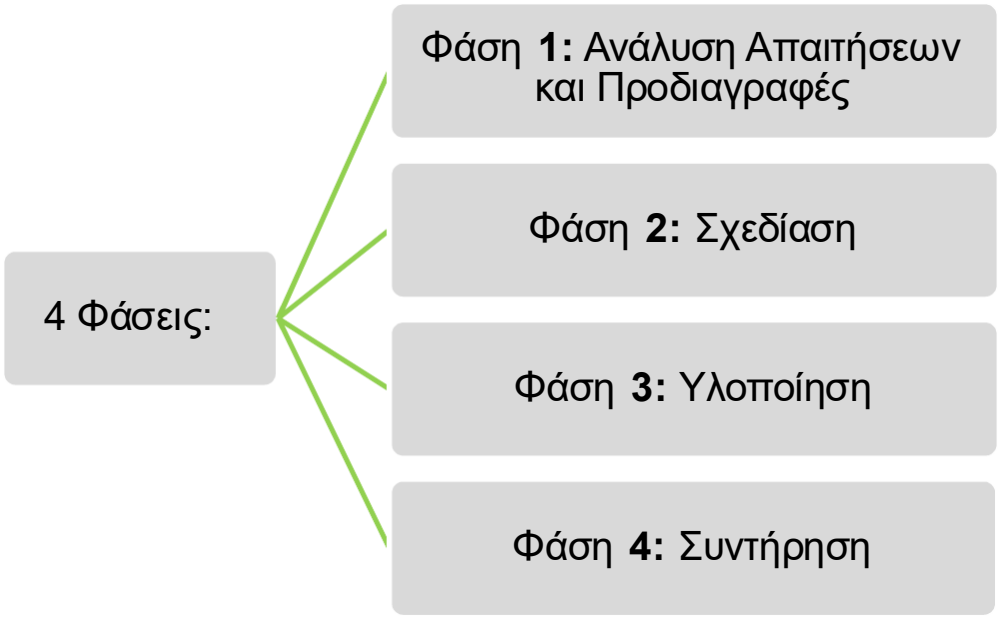
Spiral Development Model

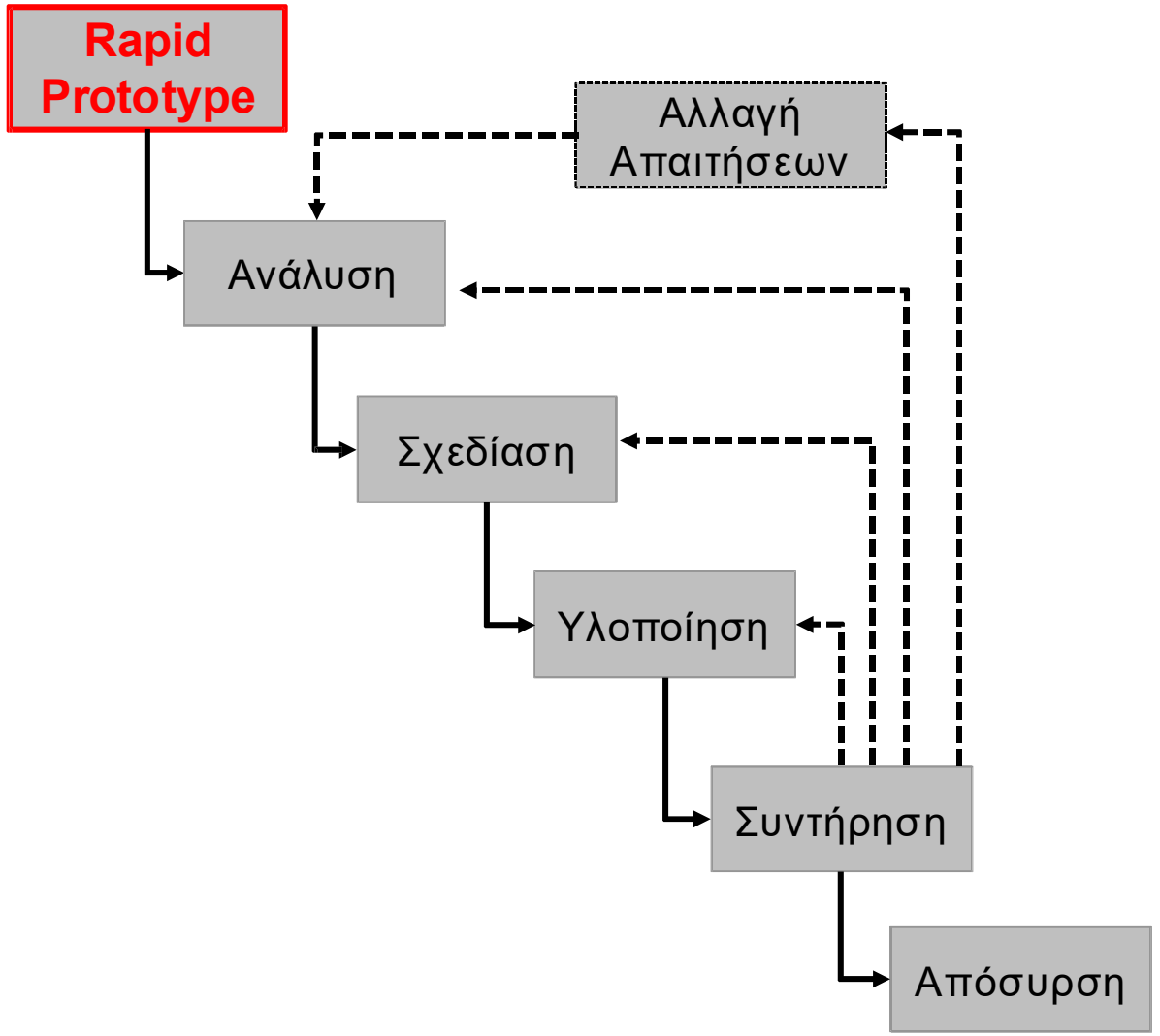


ΥΒΡΙΔΙΚΑ

...και πολλά άλλα!

Τι θα χρησιμοποιήσουμε;





Φάση 1: Ανάλυση Απαιτήσεων και Προδιαγραφές

1Α. Ανάλυση Απαιτήσεων (Requirements Analysis)

- Περιγραφή των αναγκών του πελάτη (requirements statement)
- Αναγνώριση ενδιαφερομένων
- Συνεντεύξεις χρηστών
- Ανάπτυξη περιπτώσεων χρήσης (use cases). Αποκαλύπτουν βασικά χαρακτηριστικά του προγράμματος. Αποτελούν περιγραφικές απαντήσεις σε ερωτήσεις όπως:
  - Ποιος θα χρησιμοποιήσει το σύστημα;
  - Τι ενέργειες μπορούν να κάνουν οι χρήστες;
  - Πώς αλλιώς θα μπορούσε αυτό να λειτουργήσει αν κάποιος άλλος έκανε την ενέργεια ή ο ίδιος χρήστης είχε διαφορετικό στόχο;
  - Τι προβλήματα μπορεί να προκύψουν με το σύστημα;
- Ανάπτυξη Γρήγορων Πρωτοτύπων (rapid prototyping)

Φάση 1: Ανάλυση Απαιτήσεων και Προδιαγραφές (συν.)

1Β. Προδιαγραφές Συστήματος (System Specification)

- **Προκαταρκτική αναγνώριση κλάσεων (Analysis Class Diagram)**: Συγκεκριμένη περιγραφή του τι θα πρέπει κάνει το πρόγραμμα (όχι πως θα το κάνει). Παράδειγμα του τι μπορεί να περιλαμβάνουν οι προδιαγραφές:

  - Overall description
    - Product perspective
      - System, user, hardware, software interfaces
      - Memory Constraints
      - Operations
      - Site Adaptation Requirements
    - Product functions
    - User characteristics
    - Constraints, assumptions and dependencies

  Specific requirements
  - External interface requirements
  - Functional requirements
  - Performance requirements
  - Design constraints
    - Standards compliance
  - Logical database requirement
  - Software System attributes
    - Reliability, Availability, Security, Maintainability, Portability

Φάση 2: Σχεδίαση

- Σχεδιασμός των αναγνωρισμένων κλάσεων και της αλληλεπίδρασης των κλάσεων
- Χρήση Διαγραμμάτων Ακολουθίας (sequence diagrams)
- Χρήση Διαγραμμάτων Κλάσεων (class diagrams)
- Πέντε στάδια σχεδιασμού αντικειμένων:
  - Ανακάλυψη Αντικειμένου (Object discovery)
  - Συναρμολόγηση Αντικειμένου (Object assembly)
  - Κατασκευή Συστήματος
  - Επέκταση Συστήματος
  - Επαναχρησιμοποίηση κλάσεων

Φάση 2: Σχεδίαση

- Κατευθυντήριες γραμμές για σχεδιασμό των αντικειμένων:
- Οι κλάσεις προκύπτουν από το πρόβλημα – επεκτείνονται και ωριμάζουν καθώς χρησιμοποιούνται για την επίλυση και άλλων προβλημάτων
- Το μεγαλύτερο τμήμα του σχεδιασμού σας είναι να ανακαλύψετε τις κλάσεις που σας χρειάζονται
- Δεν είναι ανάγκη να προβλέψετε τα πάντα από την αρχή – μαθαίνετε καθώς προχωράτε
- Ξεκινήστε τον προγραμματισμό νωρίς, ώστε να έχετε κάτι που θα αποδεικνύει την αξία ή προχειρότητα του σχεδιασμού σας
- KIS: keep-it-simple. Οι απλές λύσεις είναι συνήθως οι καλύτερες

Φάση 3: Υλοποίηση

- Φάση της μετατροπής του αρχικού σχεδιασμού σε πρόγραμμα προς μετάφραση και εκτέλεση

3Α. Αναγνώριση Τμήματος Πυρήνα

- Πυρήνας: Στόχος εδώ είναι να εντοπίσετε το τμήμα που αποτελεί τον πυρήνα του προγράμματός σας, από τον οποίο θα προκύψει το τελικό σύστημα
- Ο πυρήνας μπορεί να μην περιέχει τα πάντα, είναι όμως η βάση για το χτίσιμο των λειτουργιών του συστήματός σας.

Φάση 3: Υλοποίηση (συν.)

3Β. Πρόσθεση Λειτουργιών

- Μετά το κτίσιμο του πυρήνα, γίνεται η υλοποίηση των λειτουργιών του συστήματος
- Κάθε ξεχωριστή λειτουργία αντιπροσωπεύεται στη φάση τού σχεδιασμού από μια αντίστοιχη περίπτωση χρήσης (use case)
- Η υλοποίηση των διαφορετικών περιπτώσεων χρήσης γίνεται διαδοχικά – σε διαφορετικές επαναλήψεις – και μέχρι να ολοκληρωθεί η υλοποίηση όλων των συστατικών του συστήματος
- Πως δικαιολογείται η «επαναληπτική» προσέγγιση (iteration over the use cases);

Φάση 4: Συντήρηση

- Η φάση της συντήρησης ή εξέλιξης του κώδικα:
  - Διόρθωση σφαλμάτων
  - Αλλαγές στον κώδικα με βάση την εμπειρία χρήσης του
  - Πρόσθεση νέων λειτουργιών
  - Καλύτερη ικανοποίηση των απαιτήσεων

Α/Σ Σχεδιασμός και Ευθύνες

- Πως σχεδιάζουμε αντικειμενοστρεφή προγράμματα;
- Ο Α/Σ σχεδιασμός εστιάζει στην αναγνώριση των κλάσεων ενός προβλήματος και στην ανάθεση ευθυνών στις κλάσεις (λειτουργίες που η κλάση είναι υπεύθυνη να εκτελέσει)
- Η ανάθεση ευθυνών συνεπάγεται δημιουργία ανεξαρτησίας
  - Δημιουργία ανεξάρτητων υποσυστημάτων, τα οποία μπορούν εύκολα να επαναχρησιμοποιηθούν

Παράδειγμα Διαδικασίας Ανάπτυξης Λογισμικού

- Στις ακόλουθες διαφάνειες θα δούμε σαν παράδειγμα τη διαδικασία ανάπτυξης του συστήματος «Βιβλίο Διευθύνσεων»
  - Φάση 1Α: Ανάλυση Απαιτήσεων – Σενάριο, Use Cases
  - Φάση 1Β: Προδιαγραφές – Ανάλυση Κλάσεων, Αναφορά
  - Φάση 2: Σχεδιασμός – CRC, Sequence Diagrams, Class Diagram
  - Φάση 3: Υλοποίηση – Λεπτομερής Σχεδιασμός Κλάσεων, Κώδικας
  - Φάση 4: Συντήρηση - Νέες Απαιτήσεις, Διάδοση στις άλλες φάσεις

Παράδειγμα: Βιβλίο Διευθύνσεων
Φάση 1Α: Ανάλυση Απαιτήσεων

Αναφορά Απαιτήσεων

- Βιβλίο Διευθύνσεων: το πρόγραμμα αυτό **διατηρεί και συντηρεί** ένα βιβλίο διευθύνσεων.
- Το βιβλίο διευθύνσεων θα είναι μία συλλογή από οντότητες οι οποίες αναπαριστούν άτομα με πληροφορίες για όνομα, επίθετο, διεύθυνση, πόλη, επαρχία, ταχυδρομικό κώδικα και τηλέφωνο
- Το πρόγραμμα πρέπει να υποστηρίζει προσθήκη ενός ατόμου, ενημέρωση υφιστάμενου ατόμου (εκτός από το όνομα) και διαγραφή ατόμου
- Επιπρόσθετες λειτουργίες:
  - **Ταξινόμηση** των ατόμων αλφαβητικά με επίθετο, όνομα ή ταχυδρομικό κώδικα
  - **Εκτύπωση** όλων των ατόμων σε μορφή “mailing label”
  - Δημιουργία ενός καινούριου και άνοιγμα ενός υφιστάμενου βιβλίου διευθύνσεων από τον δίσκο με τη χρήση User Interface με τις συνηθισμένες επιλογές κάτω από το File (New, Open, Close, Save and Save As)

Αναφορά Απαιτήσεων (συν.)

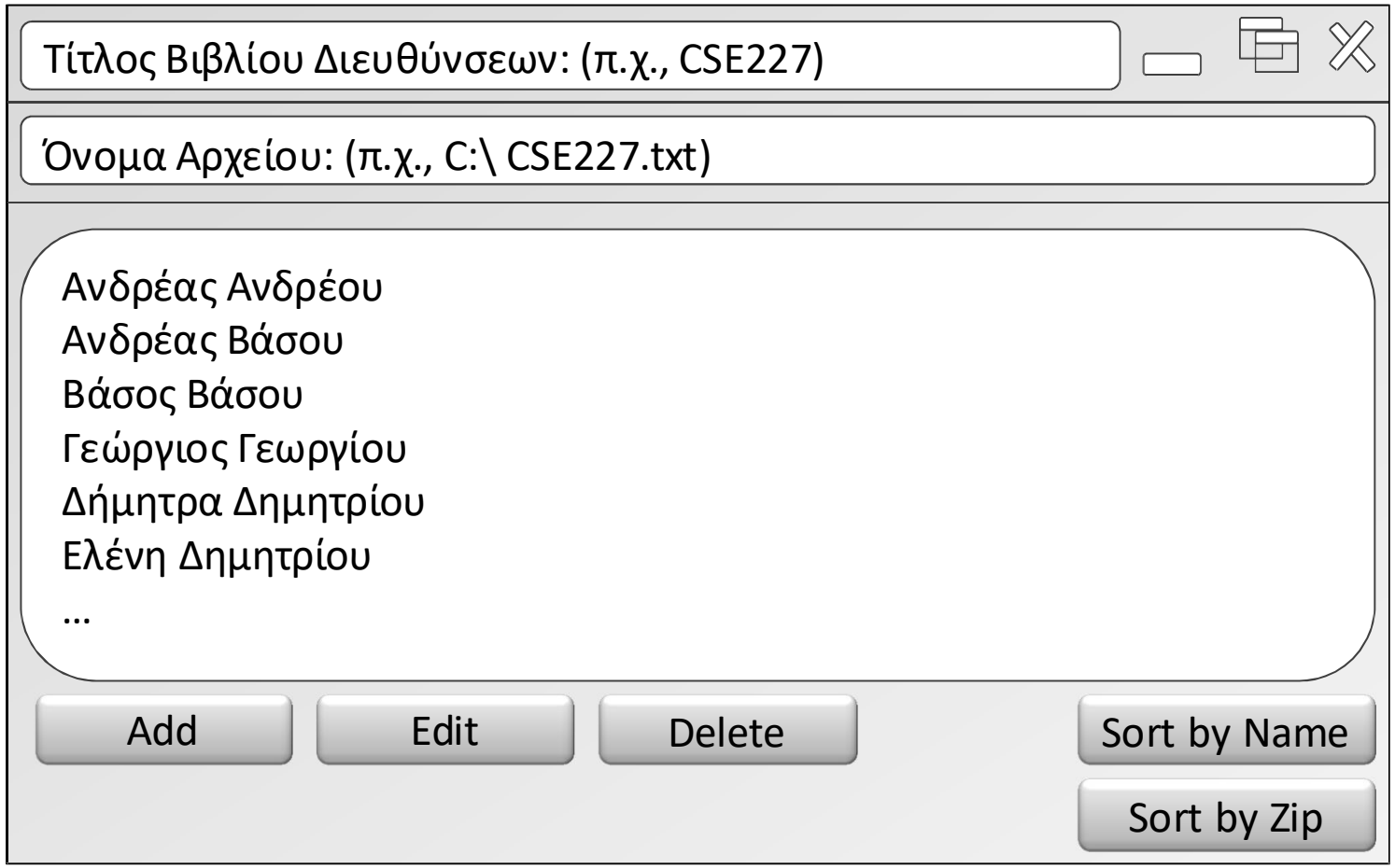
- Επιπρόσθετες λειτουργίες (συν.):
  - Το πρόγραμμα θα πρέπει να **επεξεργάζεται**, ανά πάσα στιγμή, **μόνο ένα βιβλίο διευθύνσεων**. Στην περίπτωση που ο χρήστης επιλέξει «Νέο Βιβλίο Διευθύνσεων» ή «Άνοιγμα Υφιστάμενου Βιβλίου Διευθύνσεων» τότε το πρόγραμμα θα σταματάει/ κλείνει το υφιστάμενο/τρέχον βιβλίο διευθύνσεων
  - Σε κάποια φάση στο μέλλον, το πρόγραμμα **πιθανόν να μπορεί να υποστηρίζει να τρέχουν πολλά γραμματοκιβώτια ταυτόχρονα**, το καθένα στο δικό του παράθυρο. Σε αυτή την περίπτωση, όταν ο χρήστης επιλέξει «Νέο Βιβλίο Διευθύνσεων» ή «Άνοιγμα Υφιστάμενου Βιβλίου Διευθύνσεων» τότε το πρόγραμμα δεν θα επηρεάζει τα τρέχοντα βιβλία διευθύνσεων
  - Το πρόγραμμα θα παρακολουθεί τις αλλαγές που γίνονται στο βιβλίο διευθύνσεων και θα παρέχει στο χρήστη την δυνατότητα να αποθηκεύσει τις αλλαγές στο υφιστάμενο ή σε νέο βιβλίο διευθύνσεων

Αναφορά Απαιτήσεων (συν.)

- Επιπρόσθετες λειτουργίες (συν.):
  - Το πρόγραμμα **θα παρακολουθεί το αρχείο από το οποίο έχει διαβαστεί ή που έχει αποθηκευτεί πιο πρόσφατα**, θα παρουσιάζει το όνομα του αρχείου στον τίτλο του παραθύρου και θα το χρησιμοποιεί όταν ο χρήστης επιλέξει το «Save»
  - Όταν ένα καινούριο παράθυρο δημιουργηθεί, τότε **το παράθυρο θα παρουσιάζει τον τίτλο «Untitled» και η επιλογή «Save» μετατρέπεται σε «Save As»** και ο χρήστης θα πρέπει να δώσει ένα όνομα αρχείου

Αναφορά Απαιτήσεων (συν.)

Ανάλυση Interface:

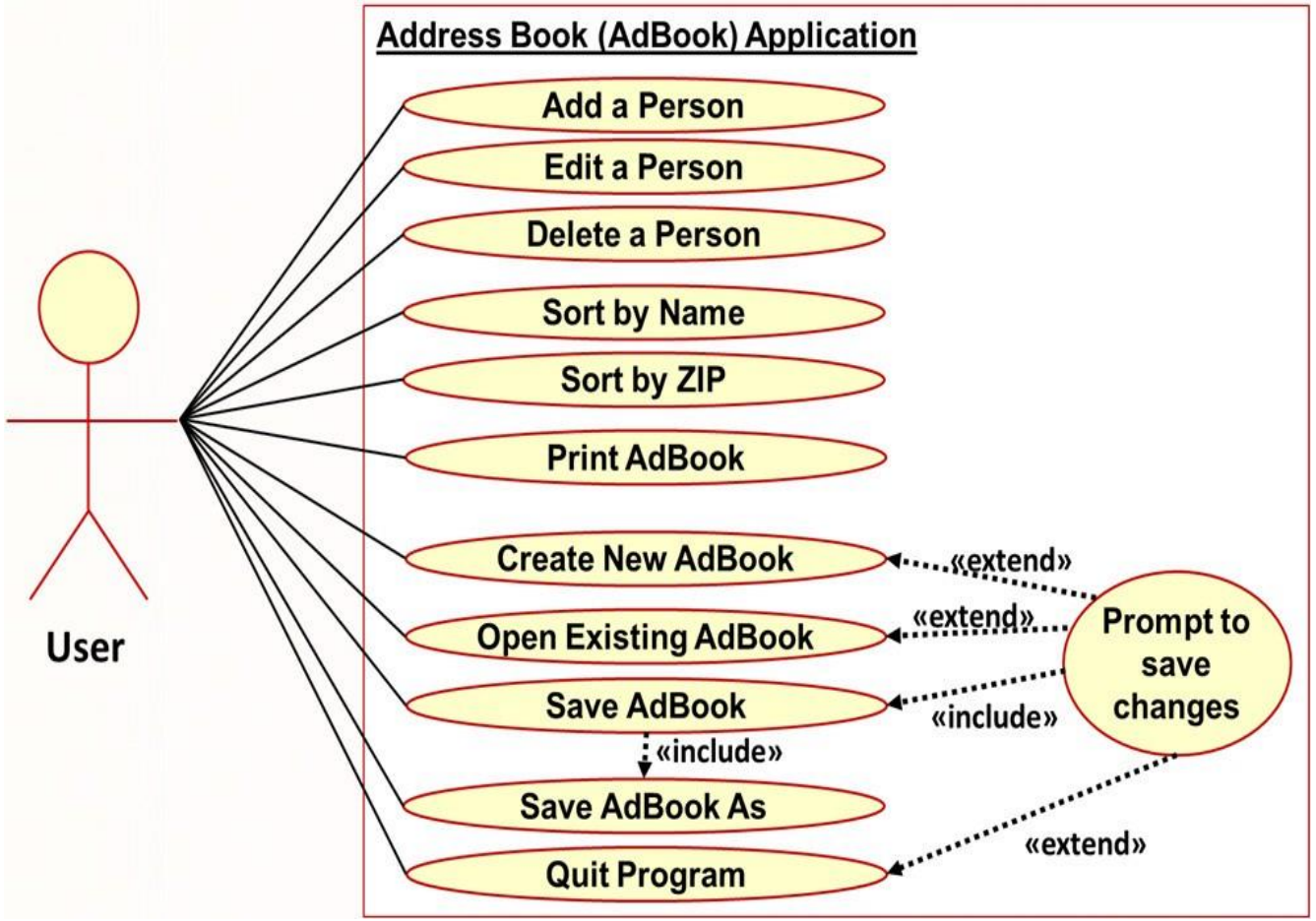


Περιπτώσεις Χρήσης (Use Cases)

- Τεχνική ανάλυσης με την οποία περιγράφουμε πώς πρέπει να λειτουργεί ένα πρόγραμμα
  - Μετάφραση της Αναφοράς Απαιτήσεων σε Use Cases
  - Κάθε use case εστιάζει σε ένα συγκεκριμένο σενάριο
  - Κάθε σενάριο περιγράφει τα **βήματα** που πρέπει να πραγματοποιηθούν για να φέρουν το σενάριο σε μία **επιτυχημένη ολοκλήρωση**
  - Η ολοκλήρωση του σεναρίου πρέπει να αντιστοιχεί στην επίτευξη ενός σημαντικού στόχου
  - Κάθε βήμα αναπαριστά μια αλληλεπίδραση του προγράμματος με ανθρώπους ή εξωτερικές οντότητες του προγραμματιστικού συστήματος

Διαγράμματα Περίπτωσης Χρήσης (Use Case Diagrams)

- **Use Case Diagram**: γραφική αναπαράσταση υψηλού επιπέδου του συστήματος
- Βλέποντας ένα use case diagram πρέπει να μπορούμε να πούμε εύκολα τι κάνει το σύστημα και ποιος αλληλοεπιδρά μαζί του
- Περιλαμβάνει τα **use cases**, που αντιπροσωπεύουν τις διάφορες λειτουργίες του συστήματος και **actors**, που αντιπροσωπεύουν τους χρήστες/ρόλους του συστήματος



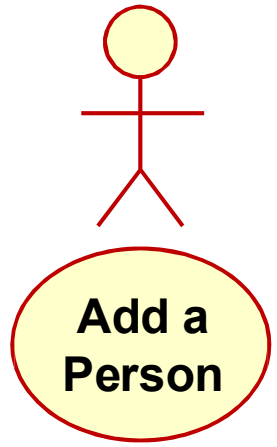
Διαγράμματα Περίπτωσης Χρήσης (Use Case Diagrams)

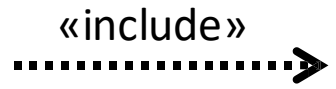
- Μπορούμε να δημιουργήσουμε περισσότερα από ένα use case diagrams για ένα σύστημα:
  - Το κάθε use case diagram θα δείχνει ένα υποσύνολο του συστήματος
  - Μπορούν να οργανωθούν σε πακέτα για καλύτερη οργάνωση του μοντέλου

- Ένα use case diagram μπορεί να βοηθήσει στην επικοινωνία με τους τελικούς χρήστες του συστήματος.
  - Είναι απλό, μη τεχνικό, ώστε ο καθένας στην ομάδα (προγραμματιστές και τελικοί χρήστες) να καταλήξουν σε μία κοινή κατανόηση του συστήματος
  - Συνήθως δημιουργείται από την τεχνική ομάδα σε συνδυασμό με αντιπρόσωπους των τελικών χρηστών

Διαγράμματα Περίπτωσης Χρήσης (Use Case Diagrams)

- Χρήση των πιο κάτω διαγραμμάτων:
  - Ρόλος/Χρήστης (actor): ένα άτομο ή ένα σύνολο από άτομα που ανήκουν στον ίδιο ρόλο και αλληλεπιδρούν με το σύστημα ή μεταξύ τους
  - Περιπτώσεις Χρήσης (use case): μία υψηλού επιπέδου λειτουργία που παρέχει το σύστημα
    - Σχέσεις (relationships): αλληλεπίδραση actor με use case
    - Σχέση «include»: υποδηλώνει ότι ένα use case ΠΡΕΠΕΙ να χρησιμοποιήσει κάποιο άλλο
    - Σχέση «extend»: υποδηλώνει ότι ένα use case επεκτείνει την βασική λειτουργία κάποιου άλλου use case
    - Σχέση «generalization»: ένας actor ή use case κληρονομεί τα χαρακτηριστικά κάποιου άλλου actor ή use case

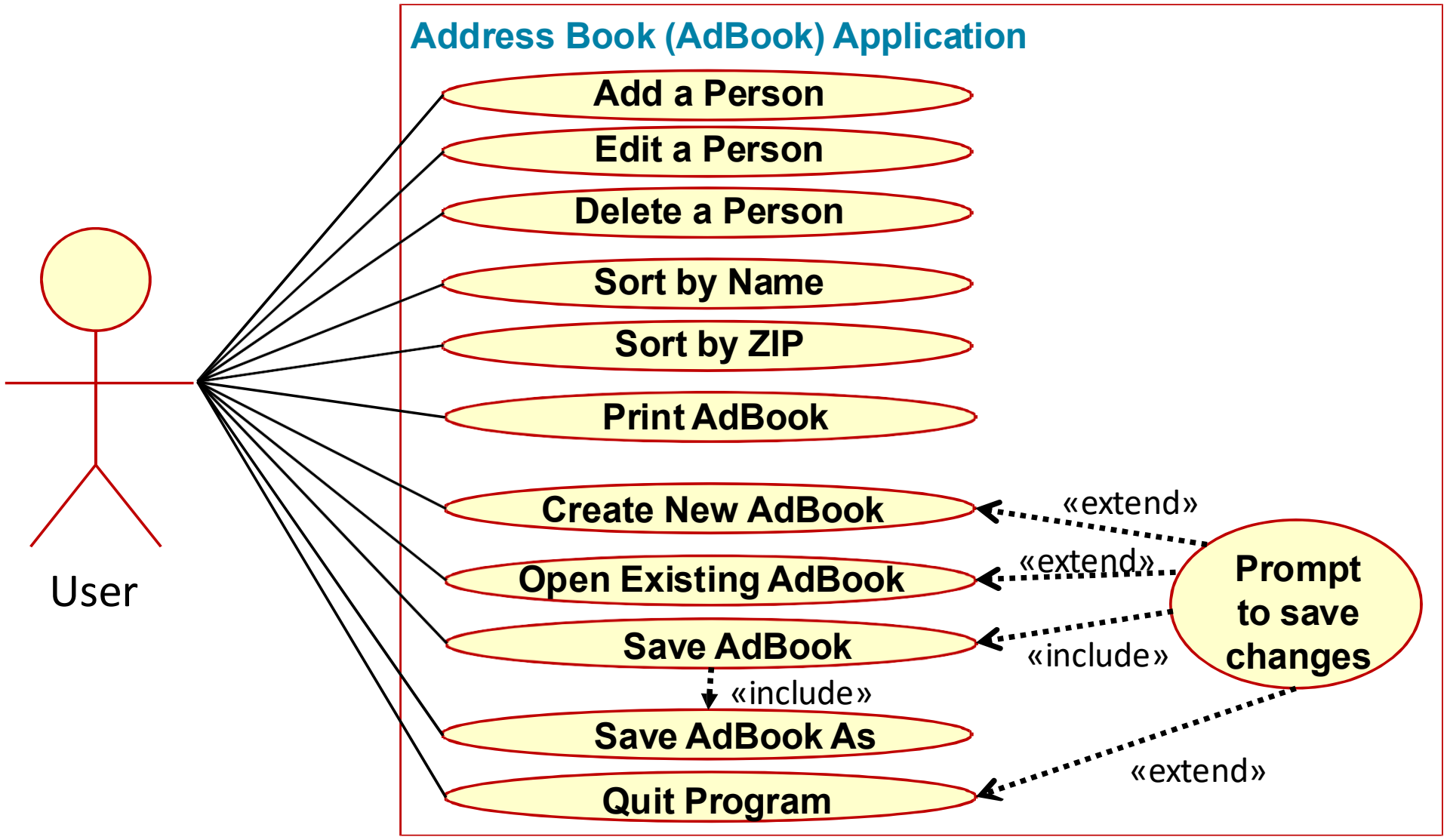




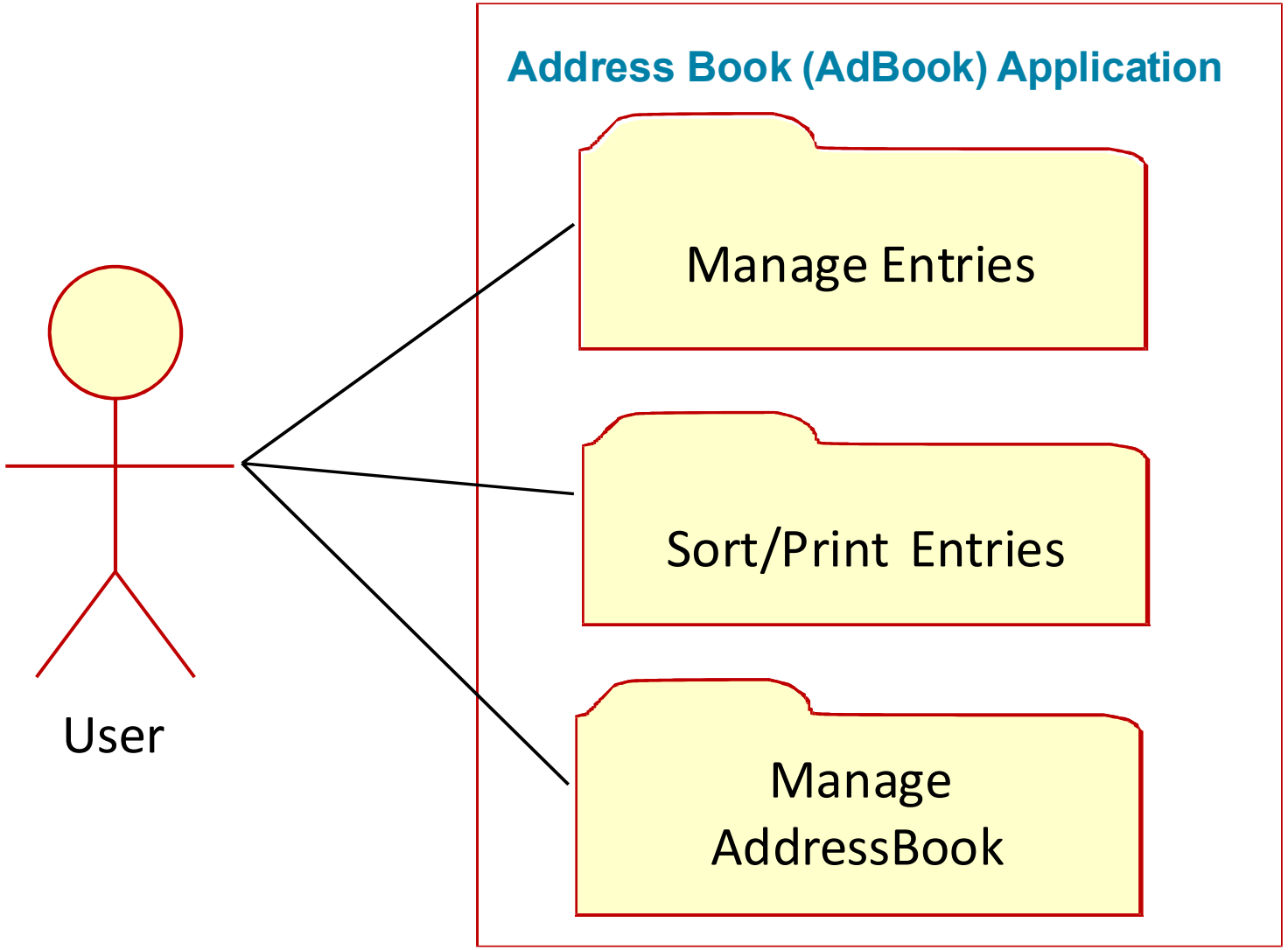




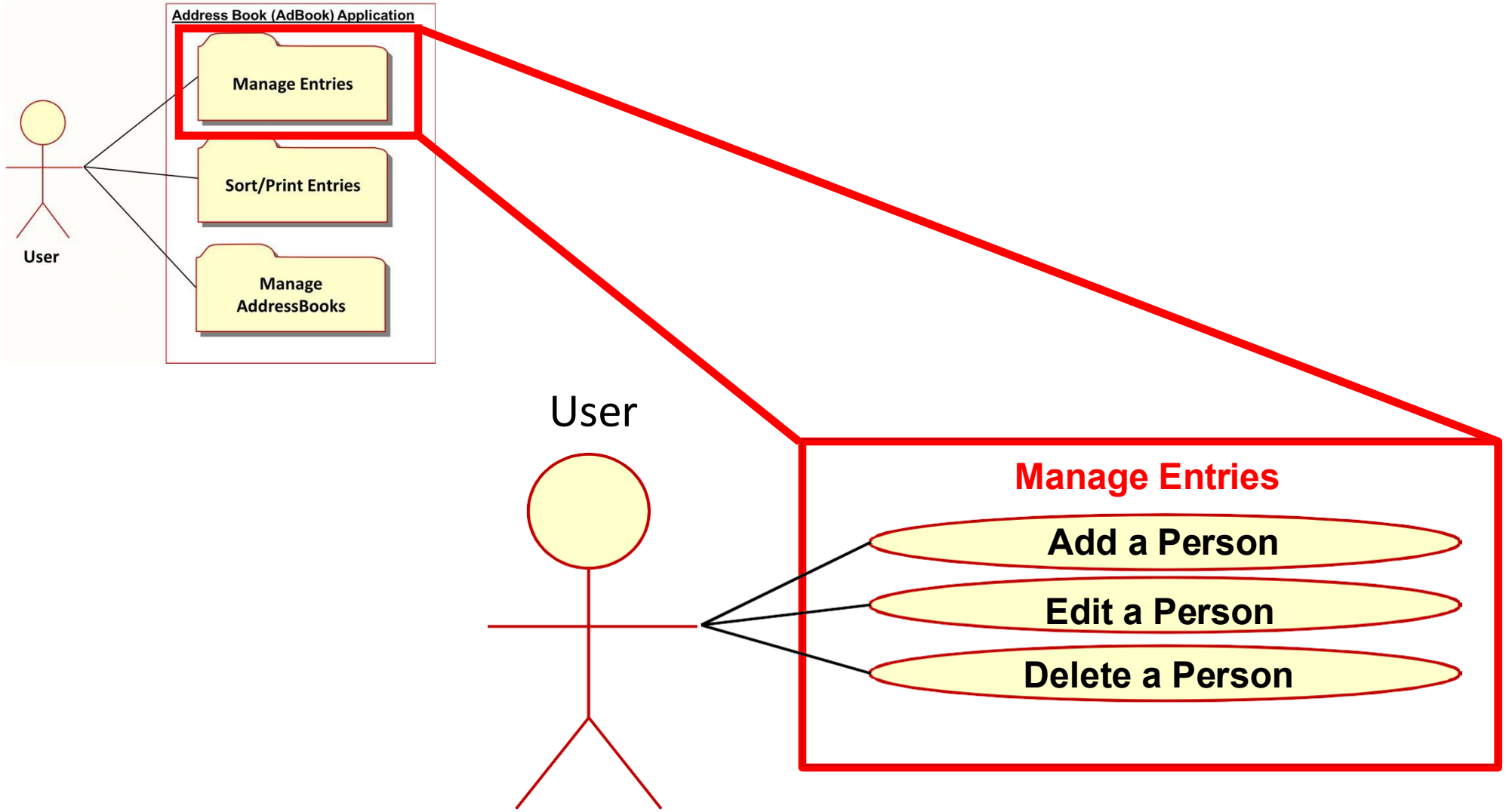
Βιβλίο Διευθύνσεων: Use Case Diagram



Βιβλίο Διευθύνσεων: Use Case Package Diagram



Βιβλίο Διευθύνσεων: Use Case Diagram (packets)



Βιβλίο Διευθύνσεων: Use Cases

- **Add a Person**
  - Το εν λόγω use case ξεκινά όταν ο χρήστης κάνει κλικ στο κουμπί ”Add a Person" στο κύριο παράθυρο.
  - Εμφανίζεται ένα παράθυρο διαλόγου, με τίτλο ”New Person", το οποίο περιέχει πεδία στα οποία πρέπει να συμπληρωθεί το όνομα και το επώνυμο του νέου ατόμου και άλλες πληροφορίες.
  - Το παράθυρο διαλόγου μπορεί να κλείσει είτε κάνοντας κλικ στο κουμπί "OK" ή ”Cancel"
  - Εάν πατηθεί το κουμπί "OK", ένα νέο άτομο προστίθεται στο τέλος του βιβλίου διευθύνσεων, και το όνομα του ατόμου, προστίθεται στο τέλος της λίστας των ονομάτων στο κύριο παράθυρο
  - Εάν πατηθεί το κουμπί "Cancel", δεν γίνονται αλλαγές ούτε στο βιβλίο διευθύνσεων, ούτε στο κύριο παράθυρο

Βιβλίο Διευθύνσεων: Use Cases

- **Edit a Person**
  - Το εν λόγω use case ξεκινά όταν ο χρήστης επιλέξει είτε ένα όνομα από τη λίστα των ονομάτων στο κύριο παράθυρο και, στη συνέχεια, κάνει κλικ στο κουμπί ”Edit", είτε κάνει διπλό κλικ σε ένα όνομα
  - Σε κάθε περίπτωση, ένα παράθυρο διαλόγου με τίτλο ”Edit «Person Name»” εμφανίζεται περιέχοντας πληροφορίες σχετικές με το επιλεγμένο πρόσωπο (εκτός από το όνομα του ατόμου, το οποίο εμφανίζεται μόνο στον τίτλο). Ο χρήστης μπορεί στη συνέχεια να επεξεργαστεί τα πεδία με τα χαρακτηριστικά του ατόμου.
  - Το παράθυρο διαλόγου μπορεί να κλείσει είτε κάνοντας κλικ στο "OK" ή "Cancel”
  - Εάν πατηθεί το κουμπί "OK", η καταχώρηση στο βιβλίο διευθύνσεων για το επιλεγμένο πρόσωπο ενημερώνεται με τις αλλαγές που έγιναν από το χρήστη
  - Εάν πατηθεί το κουμπί "Cancel", δεν γίνονται αλλαγές στο βιβλίο διευθύνσεων

Βιβλίο Διευθύνσεων: Use Cases

- **Sort by Name**
  - Το εν λόγω use case ξεκινά όταν ο χρήστης κάνει κλικ στο κουμπί “Sort by Name” στο κύριο παράθυρο
  - Οι καταχωρήσεις στο βιβλίο διευθύνσεων ταξινομούνται αλφαβητικά κατά όνομα, και ο κατάλογος στο κύριο παράθυρο ενημερώνεται ώστε να αντικατοπτρίζει τη ταξινόμηση

Βιβλίο Διευθύνσεων: Use Cases

- **Prompt to Save Changes**
  - Το εν λόγω use case ξεκινά μέσα από τη “Create New AdBook”, “Open Existing AdBook”, ή “Quit Programm”, εφόσον το βιβλίο διευθύνσεων έχει αλλάξει από τη τελευταία επιτυχημένη λειτουργία New, Open, Save, ή Save As
  - Ένα παράθυρο διαλόγου εμφανίζεται, ενημερώνοντας τον χρήστη ότι υπάρχουν μη αποθηκευμένες αλλαγές, ζητώντας από το χρήστη είτε να αποθηκεύσει τις αλλαγές, να μην αποθηκεύσει τις αλλαγές, ή να ακυρώσει τη λειτουργία
  - Αν ο χρήστης επιλέξει να αποθηκεύσει τις αλλαγές, το use case “Save” εκτελείται (το οποίο μπορεί να οδηγήσει στην εκτέλεση του use case “Save as”, σε περίπτωση που δεν υπάρχει τρέχον αρχείο)
  - Εάν ο χρήστης δεν επιλέξει να αποθηκεύσει τις αλλαγές, η αρχική λειτουργία απλά επαναλαμβάνεται. Εάν ο χρήστης επιλέξει να ακυρώσει (ή ακυρώσει το διάλογο Save), η αρχική λειτουργία ακυρώνεται

Περιγραφή Αποκλίσεων (Variations List)

- Κατά την εκτέλεσή του, ένα σενάριο μπορεί σε κάποιο στάδιο να αποτύχει
- Για παράδειγμα, στα use cases του Βιβλίου Διευθύνσεων:
  - η αποθήκευση σε ένα αρχείο μπορεί να αποτύχει γιατί το αρχείο ήδη υπάρχει και επιτρέπεται μόνο η ανάγνωση
  - η δημιουργία ενός καινούριου ατόμου μπορεί να αποτύχει γιατί δεν υπάρχει άλλη μνήμη
- Για την κάλυψη των περιπτώσεων αποτυχίας ενός σεναρίου, στην περιγραφή του use case συμπεριλαμβάνεται και η περιγραφή Αποκλίσεων (variations)
- Σε αυτό το κείμενο αναφέρονται και εναλλακτικές λύσεις για επίλυση των προβλημάτων

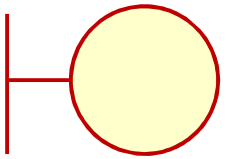
Παράδειγμα: Βιβλίο Διευθύνσεων
Φάση 1Β: Προδιαγραφές

Ανάλυση Κλάσεων (Class Analysis)

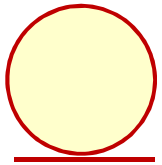
- Στα αρχικά στάδια του προγράμματος, είναι καλό να διεξαχθεί μία προκαταρκτική αναγνώριση των κλάσεων/αντικειμένων του συστήματος (Class Analysis) καθώς και οι αλληλεπιδράσεις μεταξύ τους
- Οι κλάσεις σε αυτό το προκαταρκτικό στάδιο παίρνουν την μορφή κλάσεων συνόρων, κλάσεων οντοτήτων και κλάσεων ελέγχου
- Τα use cases συνήθως αποκαλύπτουν το μεγαλύτερο ποσοστό των κλάσεων

Διαγράμματα Ανάλυσης Κλάσεων

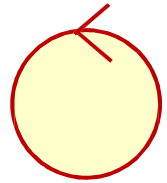
- **Κλάσεις Σύνορα (Boundary Classes)**: χειρίζονται την επικοινωνία μεταξύ **actors** και **components** του συστήματος, π.χ., user interface, device interface. Συνήθως μόνο ένα boundary class για κάθε use case



- **Κλάσεις Οντότητες (Entity Classes)**: μοντελοποιούν την πληροφορία που χειρίζεται το σύστημα



- **Κλάσεις Ελέγχου (Control Classes)**: αντιπροσωπεύουν το συντονισμό/ροή ελέγχου για ένα use case

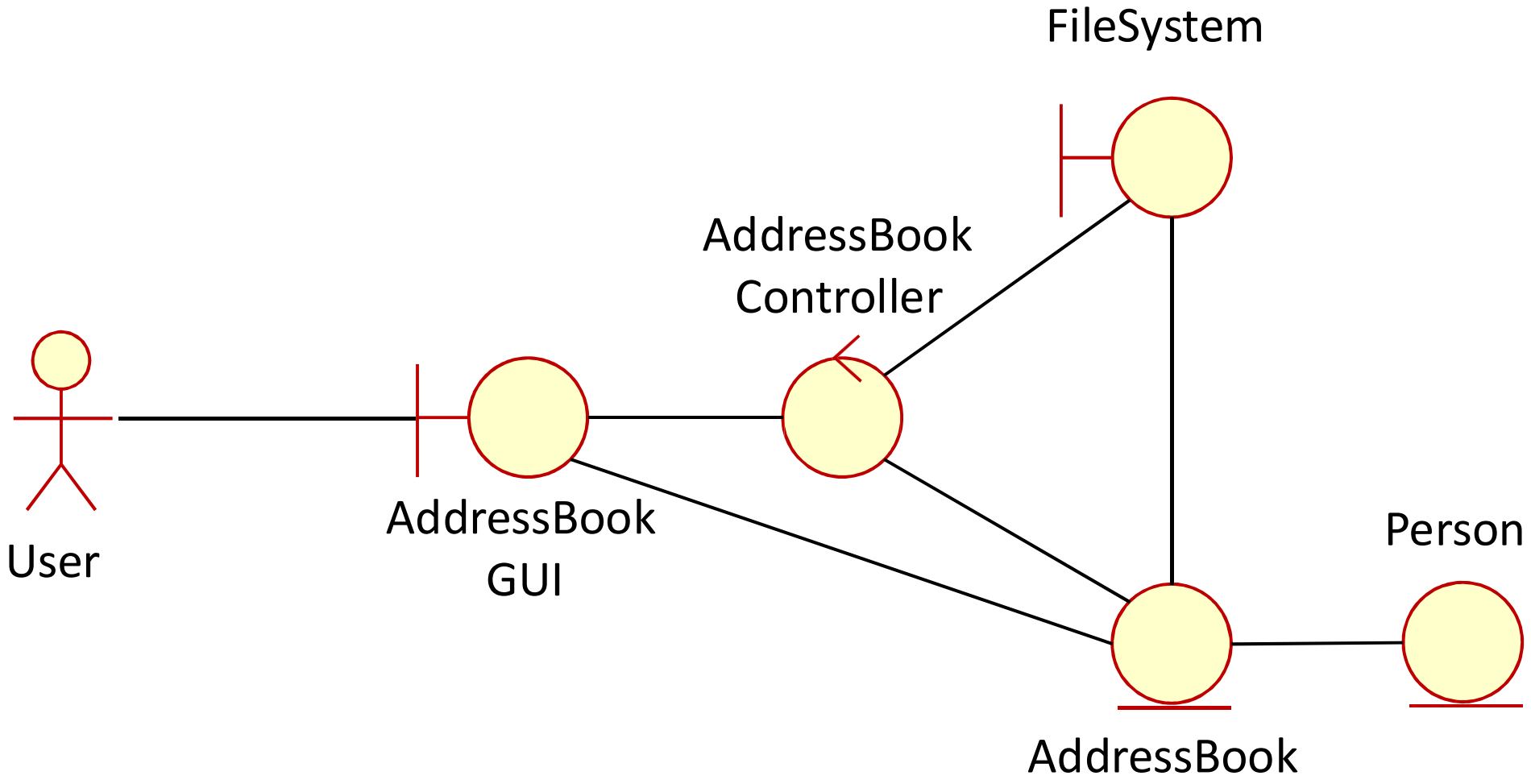


Βιβλίο Διευθύνσεων: Class Analysis

- Το πρόγραμμα, ανά πάσα στιγμή διαχειρίζεται ένα βιβλίο διευθύνσεων (AddressBook) ➔ **οντότητα**
- Κάθε άτομο (Person) μπορεί να αναπαρασταθεί σαν μία οντότητα με τα χαρακτηριστικά της ➔ **οντότητα**
- Μία διαπροσωπεία (user interface - AddressBookGUI) θα υπάρχει μεταξύ χρήστη και συστήματος. Ο χρήστης δεν θα μπορεί να έχει πρόσβαση διαφορετικά με το σύστημα ➔ **σύνορο**
- Μία διαπροσωπεία/αντικείμενο (FileSystem) θα υπάρχει μεταξύ του συστήματος και του λειτουργικού ώστε το σύστημα να μπορεί να διαχειρίζεται αρχεία ➔ **σύνορο**
- Ένα αντικείμενο διαχειριστής (AddressBookController) χρειάζεται για να αναλάβει την εκτέλεση των use cases σε ανταπόκριση των διαφόρων λειτουργιών που θα επιλέξει ο χρήστης από το μενού. ➔ **controller**

*(Για ένα τέτοιο μικρό πρόβλημα, ένας controller είναι αρκετός)

Βιβλίο Διευθύνσεων: Class Analysis Diagram



Βιβλίο Διευθύνσεων: Class Analysis (συν.)

- Το **use case** "Add a Person" έχει σχέση με την εισαγωγή νέας πληροφορίας (δηλ. νέο άτομο) από τον χρήστη. Τέλος, θα καλεστεί το αντικείμενο AddressBook ώστε να εισάξει το νέο άτομο στη συλλογή
- Το **use case** "Edit a Person" έχει σχέση με την παρουσίαση της τρέχουσας πληροφορίες κάποιου ατόμου μέσω του αντικειμένου AddressBook και παροχή της ικανότητας αλλαγή των στοιχείων αυτού. Στο τέλος, θα καλεστεί το αντικείμενο AddressBook για να αποθηκεύει τις αλλαγές
- Το **use case** "Delete a Person" έχει σχέση με την διαγραφή ενός υπάρχοντος ατόμου. Θα καλεστεί το αντικείμενο AddressBook για να αφαιρέσει το άτομο από την συλλογή
- Το **use case** "Sort by Name" θα καλέσει το αντικείμενο AddressBook για να ταξινομήσει τη συλλογή των ατόμων με το όνομα και επίθετο τους
- Το **use case** "Sort by ZIP" θα καλέσει το αντικείμενο AddressBook για να ταξινομήσει τη συλλογή των ατόμων με τον ταχυδρομικό κώδικα τους

Βιβλίο Διευθύνσεων: Class Analysis (συν.)

- Το **use case** "Create New Address Book" δημιουργεί ένα καινούριο αντικείμενο τύπου AddressBook
- Το **use case** "Open Existing Address Book" θα ζητήσει από τον χρήστη ένα αρχείο και μετά θα χρησιμοποιήσει το αντικείμενο FileSystem για να διαβάσει ένα αντικείμενο τύπου AddressBook που έχει αποθηκευτεί στο αρχείο
- Το **use case** "Save Address Book" θα χρησιμοποιήσει το αντικείμενο FileSystem για να αποθηκεύσει ένα αντικείμενο τύπου AddressBook στο πιο πρόσφατο αρχείο που έχει διαβαστεί ή αποθηκευτεί. Αν δεν υπάρχει κάποιο αρχείο τότε θα εκτελέσει το **use case** "Save Address Book As"
- Το **use case** "Print Address Book" θα ζητήσει από το αντικείμενο AddressBook να τυπώσει την συλλογή της σύμφωνα με την τρέχουσα ταξινόμηση.
- Το **use case** "Quit Program" δεν έχει σχέση με τα υπόλοιπα αντικείμενα.

Παράδειγμα: Βιβλίο Διευθύνσεων
Φάση 2: Σχεδίαση

Φάση 2: Σχεδίαση

- Σχεδιασμός των αναγνωρισμένων κλάσεων και της αλληλεπίδρασης των κλάσεων
- Χρήση της τεχνικής των Καρτών - Class-Responsibilities-Collaborators (CRC)
- Χρήση Διαγραμμάτων Ακολουθίας (sequence diagrams)
- Χρήση Διαγραμμάτων Κλάσεων (class diagrams)
- Πέντε στάδια σχεδιασμού αντικειμένων:
  - Ανακάλυψη Αντικειμένου (Object discovery)
  - Συναρμολόγηση Αντικειμένου (Object assembly)
  - Κατασκευή Συστήματος
  - Επέκταση Συστήματος
  - Επαναχρησιμοποίηση κλάσεων

Κάρτες CRC (ΚΛΑΣΕΙΣ-ΕΥΘΥΝΕΣ-ΣΥΝΕΡΓΑΤΕΣ)

- Στη μέθοδο των καρτών CRC προκρίνεται:
  - Η διερεύνηση των «υπευθυνοτήτων» των κλάσεων, αποφεύγοντας την περιγραφή δομών δεδομένων και αλγοριθμικών διεργασιών
  - Η χρήση απλών χάρτινων καρτών Class-Responsibility-Collaboration (CRC) 10cm x 15cm αντί για διαγράμματα, μοντέλα, ή προσδιορισμό μεθόδων και interface
  - Ο συμμετοχικός σχεδιασμός
  - Υπευθυνότητα είναι μια υψηλού επιπέδου περιγραφή της αποστολής μιας κλάσης
- Με μια CRC κάρτα μπορούμε να απεικονίσουμε μια κλάση και τα χαρακτηριστικά της, στη φάση της σχεδίασης

Κάρτες CRC (συν.)

Class name	Super class
Sub Classes Κλάσεις οι οποίες κληρονομούν από την κλάση	
Responsibilities Κατάλογος με τις ευθύνες (λειτουργίες) αυτής της κλάσης	Collaborators Κλάσεις με τις οποίες συνεργάζεται η κλάση

Κάρτες CRC: Περιγραφή Υπευθυνοτήτων

- Ο χώρος των **responsibilities** είναι το συμβόλαιο με το οποίο καθορίζονται τα χαρακτηριστικά και οι ευθύνες μιας κλάσης
- Οι ευθύνες περιγράφουν το πρόβλημα που πρέπει να επιλύει η κλάση:
  - χρησιμοποιώντας σύντομα ρήματα και μικρές εκφράσεις
  - περιγράφοντας το τι πρέπει να γίνει και όχι το πως θα γίνει
  - βασική επεξήγηση: ποιό θα είναι το αποτέλεσμα αν χρησιμοποιηθεί το αντικείμενο κατά τον Α ή Β τρόπο
  - το πολύ μια κάρτα ανά κλάση

Κάρτες CRC: Περιγραφή Συνεργατών

- Κατάλογος των κλάσεων με τις οποίες **“συνεργάζεται”** η παρούσα κλάση, δηλ. οι κλάσεις, τις οποίες πρέπει να λαμβάνει υπόψη της:
  - Κλάσεις που παρέχουν υπηρεσίες στην κλάση ώστε αυτή να μπορεί να διεκπεραιώσει τις ευθύνες της
  - Κλάσεις που χρησιμοποιούν υπηρεσίες της παρούσας κλάσης
- Η απόφαση για συμπερίληψη στον κατάλογο συνεργατών καθορίζεται από το βαθμό συνεργασίας μεταξύ δύο κλάσεων. Π.χ., η κλάση «στοίβα» δεν συμπεριλαμβάνει τις κλάσεις που πιθανώς να την χρησιμοποιούν (όχι όμως το αντίστροφο)

Σχεδιασμός με Σενάρια

- Ο σχεδιασμός με κάρτες CRC δεν ακολουθεί ούτε το top-down ούτε το bottom-up μοντέλο σχεδιασμού λογισμικού
- Διεργασία σχεδίασης με CRC:
  - Από το γνωστό προς το άγνωστο. Ξεκινάμε από τις προφανείς κλάσεις κι αυτές που απαιτούνται για το ξεκίνημα της εφαρμογής
  - Συνεχίζουμε ξετυλίγοντας υποθετικά σενάρια (“what if”)
  - Χρησιμοποιούμε τις κάρτες για να προσομοιώσουμε την εξέλιξη του προγράμματος
  - Κάθε ενέργεια που αναγνωρίζεται, καταχωρείται σαν υπευθυνότητα της σχετικής κλάσης

Σχεδιασμός με Σενάρια (συν.)

- Δοκιμάζονται διαφορετικά σενάρια και δημιουργούνται νέες υπευθυνότητες
- Ελέγχονται σενάρια που εμπεριέχουν καταστάσεις σφάλματος ή έκτακτα περιστατικά
- Οι σχεδιαστές ενθαρρύνονται να εργάζονται με το παρόν πρόβλημα και να μην προσπαθούν να προβλέψουν μελλοντικές απαιτήσεις του λογισμικού, οι οποίες δεν προκύπτουν από τα εξεταζόμενα σενάρια
- Συχνά, οι υπευθυνότητες αλλάζουν κλάση, καθώς εξελίσσεται η σχεδίαση και εξετάζονται διαφορετικά σενάρια

Πλεονεκτήματα των CRC

- Ως τεχνική σχεδιασμού είναι:
  - απλή
  - φθηνή
  - οι κάρτες αντικαθιστούνται εύκολα
  - μεταβάλλονται εύκολα

- Με την χρήση και τον έλεγχο σεναρίων βεβαιωνόμαστε ότι το σχέδιο του λογισμικού περιέχει όλες τις πληροφορίες, οι οποίες είναι αναγκαίες για την εκτέλεση και την εκπλήρωση μιας εργασίας

Σχεδιασμός με CRC

- Η τεχνική CRC χωρίζεται σε δύο μέρη:
- Το ερευνητικό μέρος (exploratory phase)
  - ανακαλύπτουμε τις κλάσεις που χρειάζονται για υλοποίηση του προβλήματος
  - καθορίζουμε και αναθέτουμε τις υπευθυνότητες στις διάφορες κατηγορίες
  - αναγνωρίζουμε την συνεργασία που υπάρχει μεταξύ των διαφόρων κατηγοριών
- Το αναλυτικό μέρος (analysis phase)
  - εκλέπτυνση του σχεδίου που προέκυψε από το ερευνητικό μέρος
  - δημιουργία κλάσεων που περιέχουν κοινές ευθύνες (ιεραρχία)
  - τοποθέτηση σχετικών κλάσεων μαζί

Βιβλίο Διευθύνσεων: Κάρτες CRC

FileSystem	
Responsibilities - Διάβασε ένα αποθηκευμένο βιβλίο διευθύνσεων από το αρχείο - Αποθήκευσε ένα βιβλίο διευθύνσεων σε αρχείο	**Collaborators** - AddressBook - AddressBook

Βιβλίο Διευθύνσεων: Κάρτες CRC

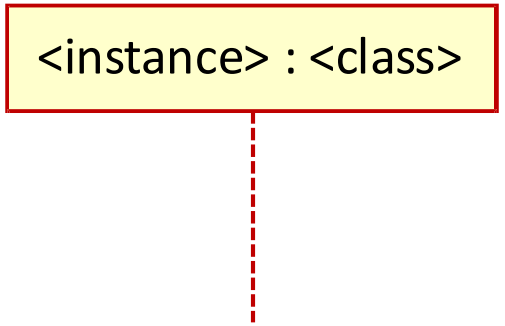
Person	
Responsibilities - Δημιουργία καινούριου ατόμου με τα στοιχεία του - Επιστροφή στοιχείων ατόμου - Ενημέρωση στοιχείων ατόμου (εκτός ονόματος)	**Collaborators**

Διαγράμματα Ακολουθίας (sequence diagrams)

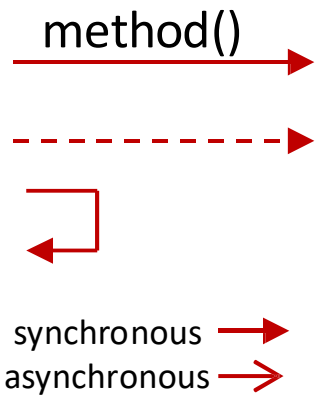
Διαγράμματα Ακολουθίας (sequence diagrams):

- Περιγράφουν τις αλληλεπιδράσεις μεταξύ αντικειμένων κατά την εκτέλεση σεναρίου (use case) σε σχέση με το χρόνο
- Παρουσιάζουν την ροή της λειτουργίας και του ελέγχου ανάμεσα στα αντικείμενα του συστήματος ή use case
- Απαντούν ερωτήσεις για το πώς δουλεύει το σύστημα
- Μπορούν να υπάρχουν πολλαπλά διαγράμματα ακολουθίας για ένα use case

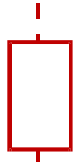
Σχεδίαση Διαγραμμάτων Ακολουθίας



- Γραμμή Ζωής (lifeline) : αντιπροσωπεύουν ρόλους ή αντικείμενα (στιγμιότυπα κλάσης) που λαμβάνουν μέρος στην συγκεκριμένη ακολουθία. Μέσα στο κουτί αναγράφεται το όνομα του στιγμιότυπου και το όνομα της κλάσης

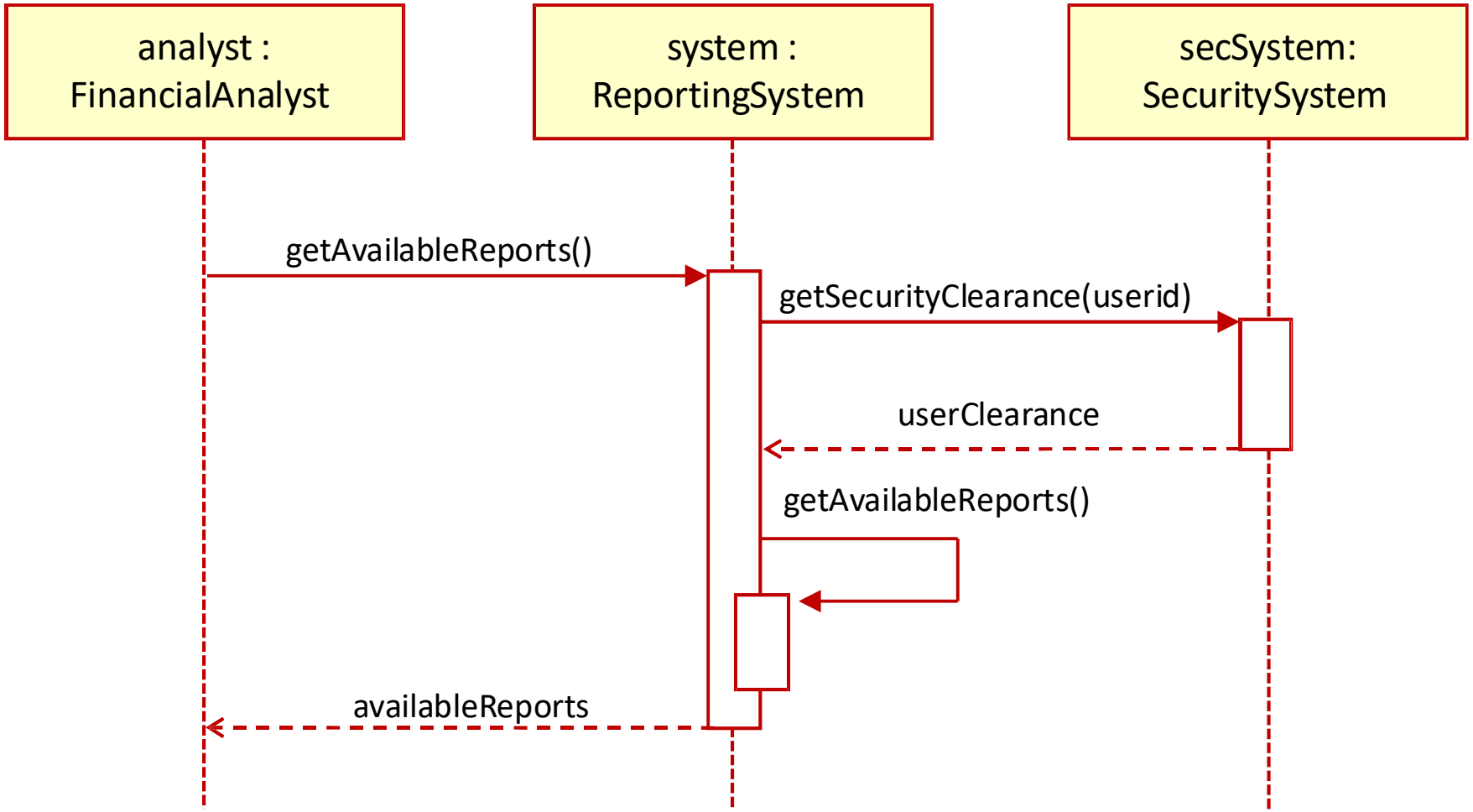


- Μηνύματα (messages): Ένα αντικείμενο καλεί μία λειτουργία κάποιου άλλου αντικειμένου.
  - Μηνύματα επιστροφής (dashed arrow)
  - Μηνύματα από ένα αντικείμενο στον εαυτό του
  - Σύγχρονα (το αντικείμενο περιμένει μέχρι να τελειώσει η λειτουργία) και ασύγχρονα μηνύματα (το αντίστροφο)



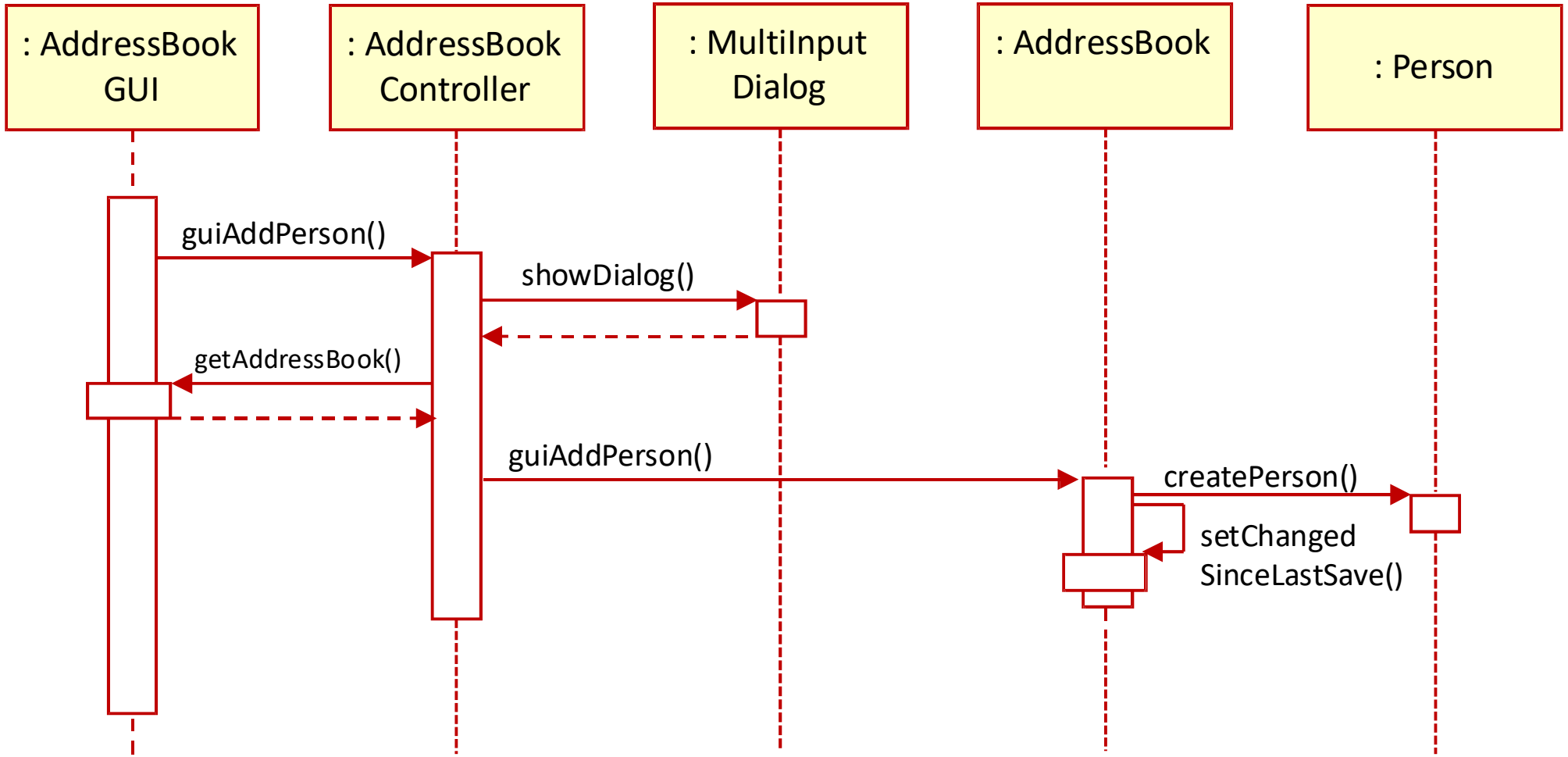
- Στήλη ενεργοποίησης (activation bar): αντιπροσωπεύει τη ζωή μίας λειτουργίας. Παρουσιάζεται μαζί με τη γραμμή ζωής.

Παράδειγμα: Sequence Diagrams



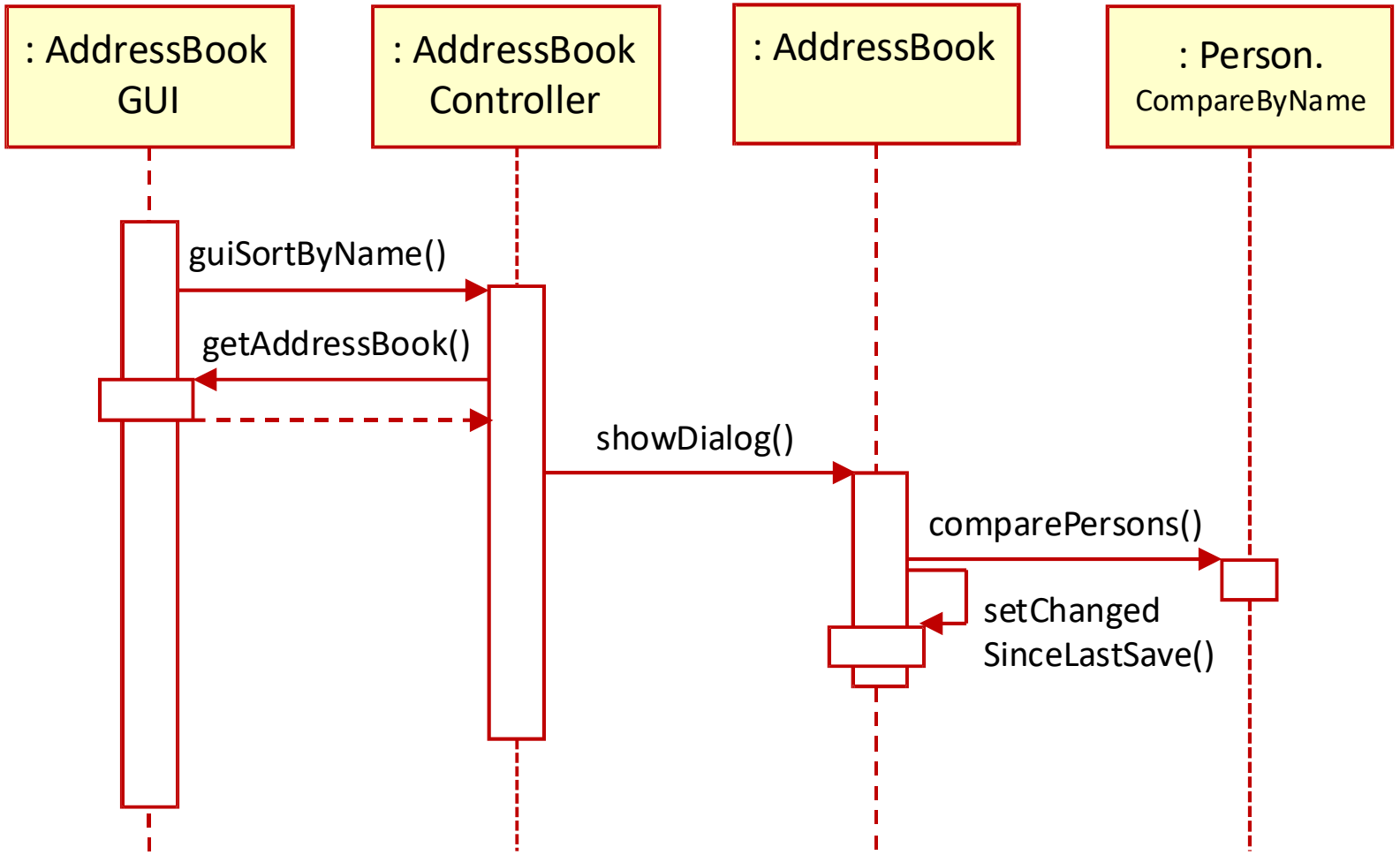
Βιβλίο Διευθύνσεων: Sequence Diagrams

Use case: Add Person



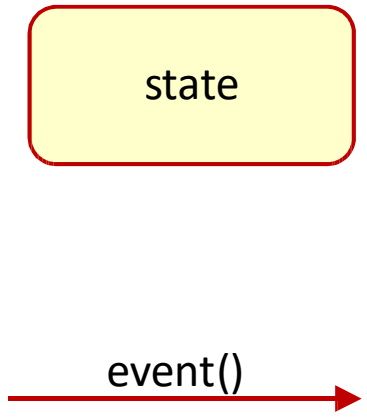
Βιβλίο Διευθύνσεων: Sequence Diagrams

Use case: Sort by Name



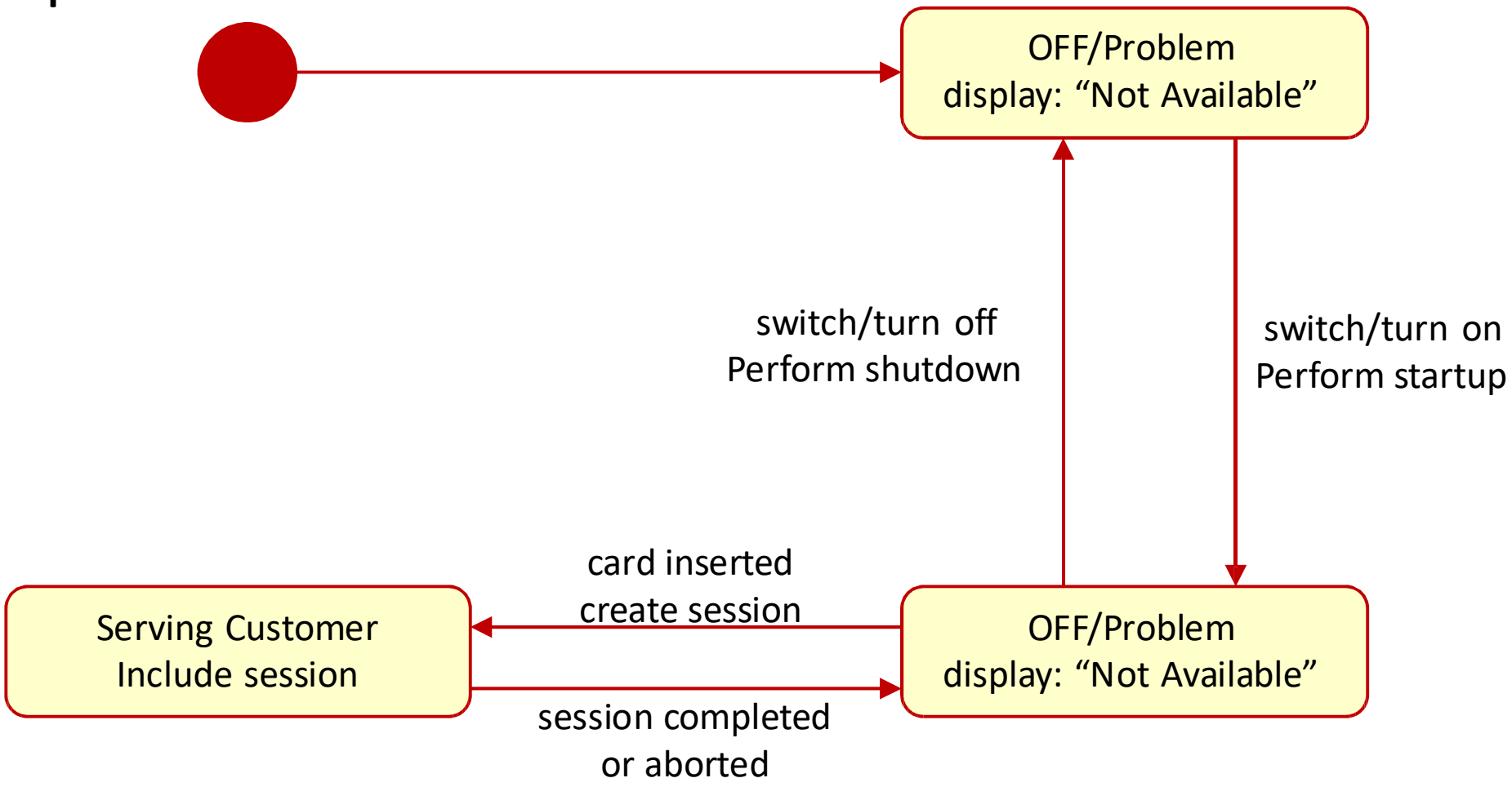
Διαγράμματα Κατάστασης (state diagrams)

- Απλά διαγράμματα τα οποία παρουσιάζουν μία μοντελοποίηση των καταστάσεων που μπορεί να βρεθεί το σύστημα ή use case:
- Προϋποθέτουν ότι ένα σύστημα μπορεί να περιγραφεί με ένα διακριτό αριθμό βημάτων
- Υπάρχουν πολλές μορφές
- Περιλαμβάνουν:
  - Καταστάσεις (states): αντιπροσωπεύουν καταστάσεις του συστήματος ή κάποιου αντικειμένου
  - Συμβάν (event): Μία λειτουργία που διαδέχεται μεταξύ καταστάσεων

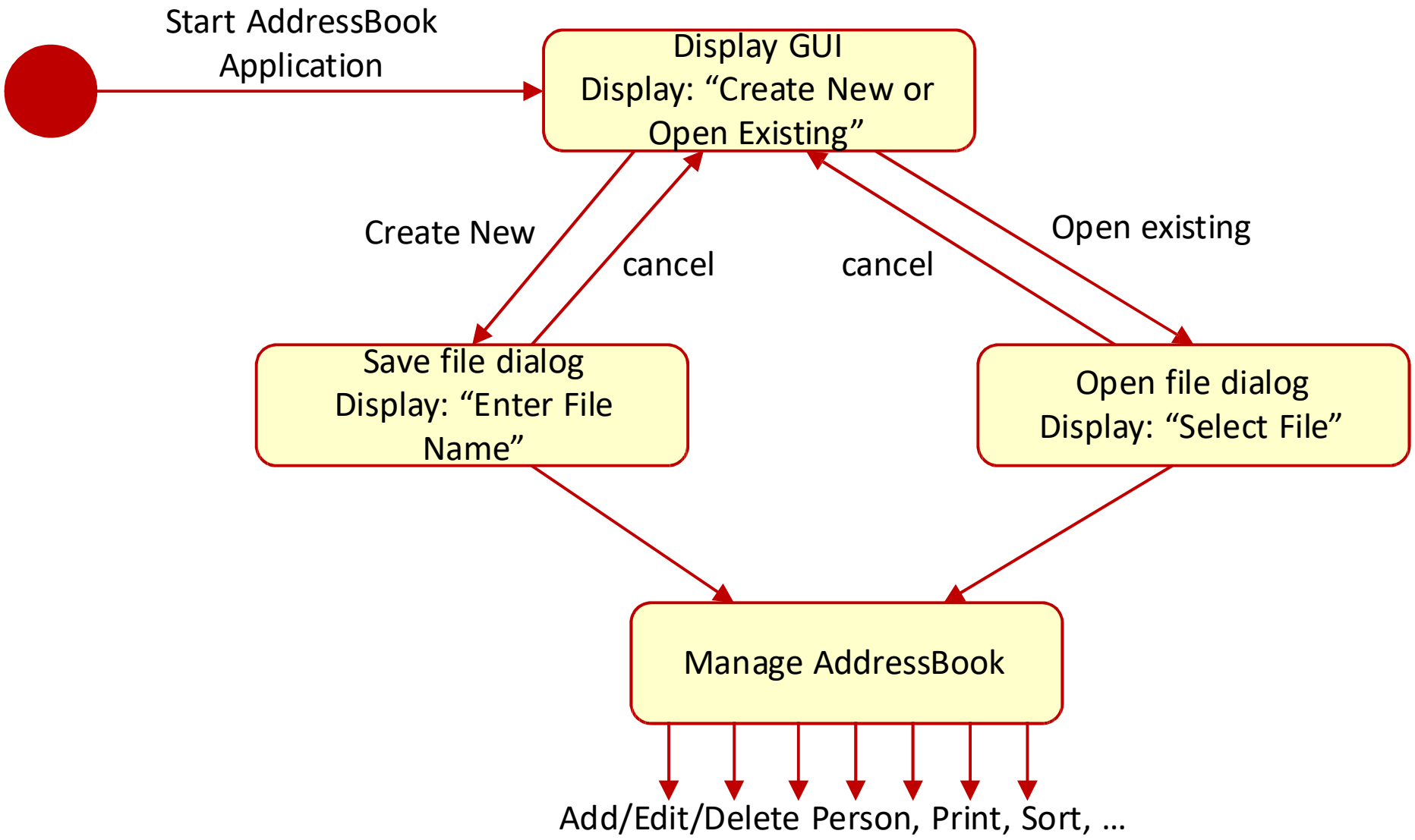


Παράδειγμα: state diagram

Μηχανή ATM



Βιβλίο Διευθύνσεων: state diagram



Διαγράμματα Κλάσης (class diagrams)

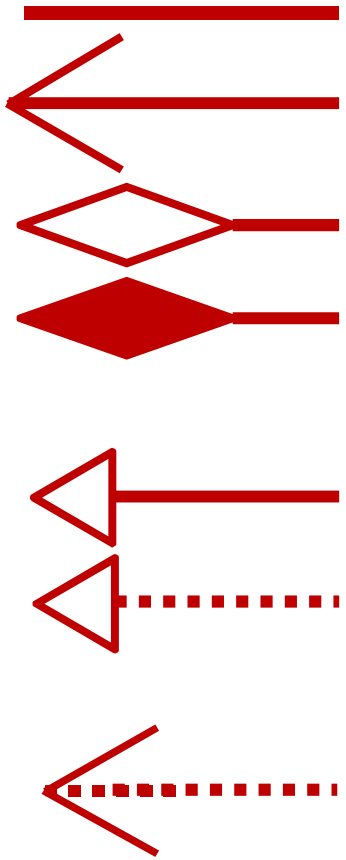
- Ένα από τα κυριότερα εργαλεία αντικειμενοστραφούς μοντελοποίησης
- Περιγράφει τη δομή του συστήματος παρουσιάζοντας:
  - τις κλάσεις του συστήματος
  - τις σχέσεις μεταξύ κλάσεων
- Δύο είδη διαγραμμάτων:
  - Γενικό εννοιολογικό μοντέλο (**ΓΜ**): χρησιμοποιείται στην ανάλυση και σχεδίαση
  - Αναλυτικό μοντέλο (**ΑΜ**): χρησιμοποιείται για μετάφραση των μοντέλων σε πραγματικό κώδικα (θα το κοιτάξουμε στη Φάση 3: υλοποίηση)

Στοιχεία Διαγράμματος Κλάσης

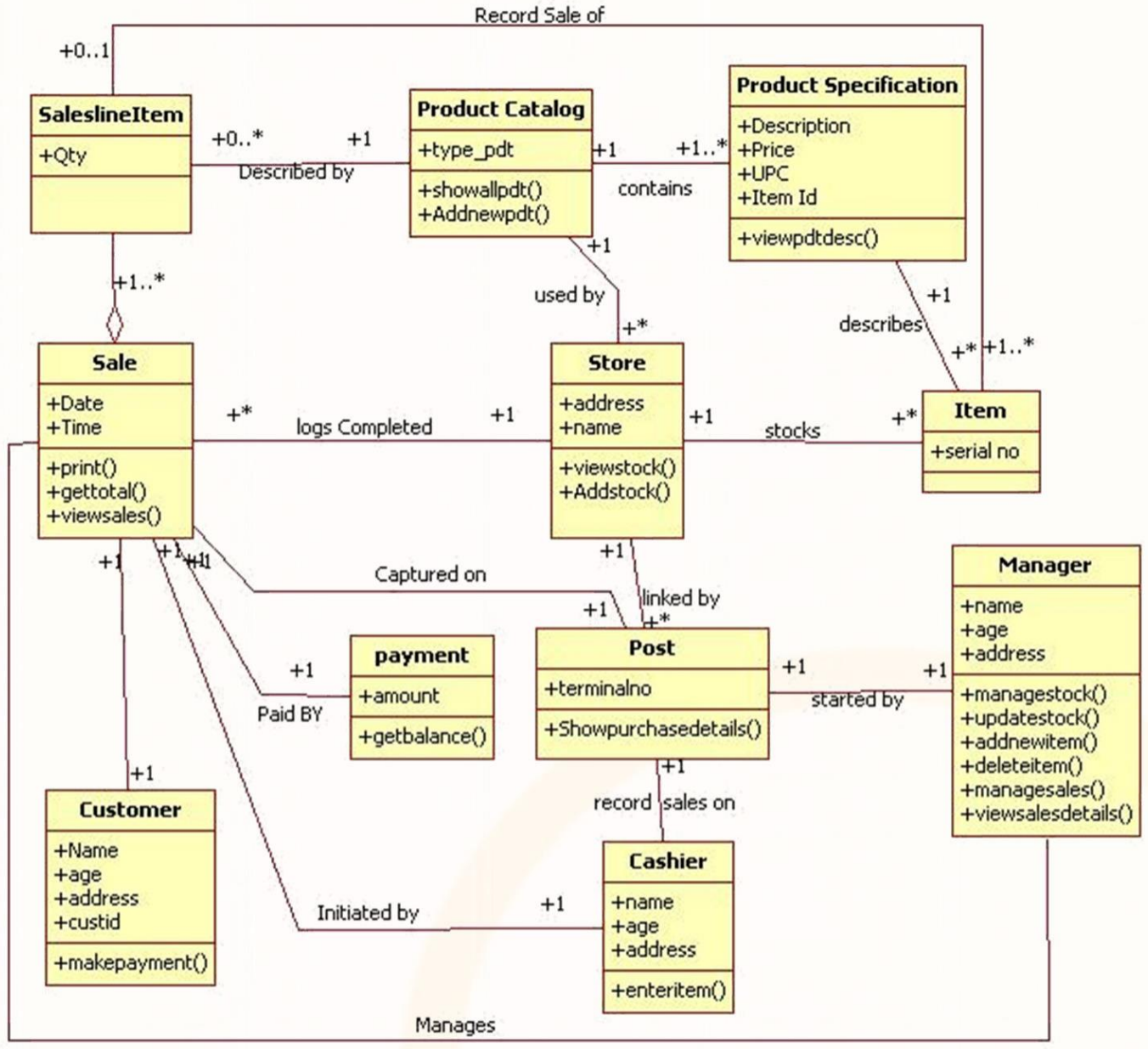
- Κλάσεις (Classes)
  - Ένα όνομα το οποίο αναπαριστά κάποια οντότητα (ΓΜ+ΑΜ)
  - Ένα σύνολο από χαρακτηριστικά που σχετίζονται με την κλάση δηλ., κάθε αντικείμενο της κλάσης θα έχει αυτά τα χαρακτηριστικά (ΑΜ)
  - Ένα σύνολο από μεθόδους που θα παρέχουν τα αντικείμενα της κλάσης (ΑΜ)
- Σχέσεις (Relationships)
  - Υποδεικνύουν τις σχέσεις μεταξύ κλάσεων (ΓΜ+ΑΜ)
  - Υποδεικνύουν τις σχέσεις μεταξύ στιγμιότυπων μίας κλάσης (ΓΜ+ΑΜ)
  - Υποδεικνύουν γενικές σχέσεις μεταξύ κλάσεων και αντικειμένων (ΓΜ+ΑΜ)
- Πολλαπλότητες
  - Επιβάλλουν ποσοτικούς περιορισμούς στις σχέσεις μεταξύ κλάσεων (ΓΜ+ΑΜ)

Σχέσεις μεταξύ κλάσεων

- Σχέσεις Επίπεδο Στιγμιότυπου (Instance-level relationships)
  - Συσχέτιση (Association)
  - Κατευθυνόμενη Συσχέτιση (Directed Association)
  - Συνάθροιση(Aggregation)
  - Σύνθεση (Composition)
- Σχέσεις σε Επίπεδο Κλάσης (Class-level relationships)
  - Γενίκευση (Generalization)
  - Πραγματοποίηση (Realization)
- Γενικές Σχέσεις (General relationships)
  - Εξάρτηση (Dependency)
  - Πολλαπλότητα (Multiplicity)



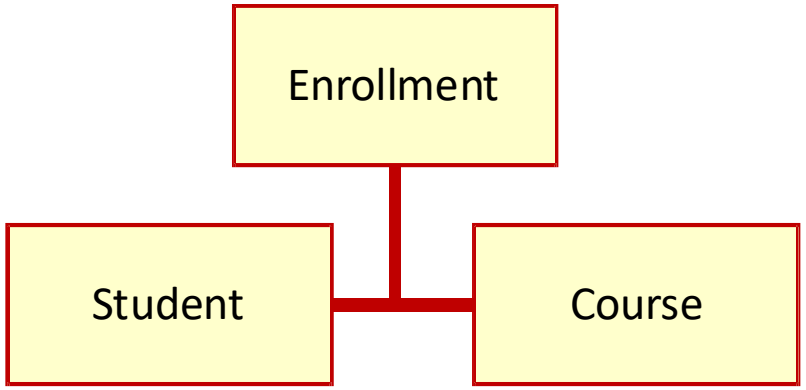
Class Diagram Example



Σχέση: Συσχέτιση (Association)

- Υποδεικνύει ότι μία κλάση χρειάζεται να επικοινωνήσει με μία άλλη κλάση
  - Μόνιμη σχέση
  - Η κατάσταση της κλάσης Α περιέχει την κλάση Β

- Παράδειγμα:
  - Ο χρήστης για να γραφτεί σε ένα μάθημα πρέπει να περάσει από τη διαδικασία της εγγραφής (enrolment)
  - Οι φοιτητές έχουν σχέση με τα μαθήματα
  - Τα μαθήματα έχουν σχέση με φοιτητές



Σχέση: Κατευθυνόμενη Συσχέτιση (Directed Association)



- Παρόμοια με σχέση συσχέτιση (association)
- Υποδεικνύει ότι μία κλάση Α γνωρίζει για μία κλάση Β αλλά η κλάση Β δεν γνωρίζει για την κλάση Α

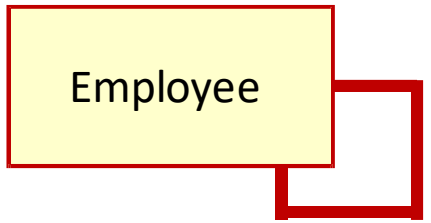
- Παράδειγμα:
  - Το FBI γνωρίζει πληροφορίες για τους πολίτες. Οι πολίτες όμως δεν γνωρίζουν για το FBI.



Σχέση: Ανακλαστική Συσχέτιση (Reflexive Association)

- Παρόμοια με σχέση συσχέτισης (association)
- Υποδεικνύει ότι μία κλάση έχει σχέση με τον εαυτό της
- Επίσης μπορεί να υποδηλώνει ότι μία κλάση έχει πολλαπλές ιδιότητες

- Παράδειγμα:
  - Κάθε υπάλληλος μπορεί να έχει διευθυντή
  - Κάθε υπάλληλος μπορεί να είναι διευθυντής, υπάλληλος ταμείου, διαφημιστής, λογιστής κτλ.



Σχέση: Συνάθροιση (Aggregation)



- Προκύπτει όταν αντικείμενα μιας κλάσης περιέχουν αντικείμενα κάποιας άλλης κλάσης για μια χρονική περίοδο
- Ειδική κατηγορία συσχέτισης-εξάρτησης (“has-a”)
- Ένα πεδίο δεδομένων μιας κλάσης μπορεί να περιγραφεί είτε με συνάθροιση είτε σαν attribute της κλάσης.

- Παράδειγμα:
  - Ένα τιμολόγιο περιέχει 0 ή περισσότερες γραμμές προϊόντων



Σχέση: Σύνθεση (Composition)



- Πιο ισχυρή μορφή συνάθροισης
- Τα συναθροισμένα αντικείμενα δεν έχουν ανεξάρτητη ύπαρξη έξω από το αντικείμενο που τα συναθροίζει
- Χρήση συνάθροισης ή σύνθεσης μόνο όταν η μια κλάση διαχειρίζεται τα αντικείμενα της άλλης
- Ειδική κατηγορία συσχέτισης εξάρτησης ("part-of")

- Παράδειγμα:
  - Μία μηχανή είναι μέρος κάποιου αυτοκινήτου

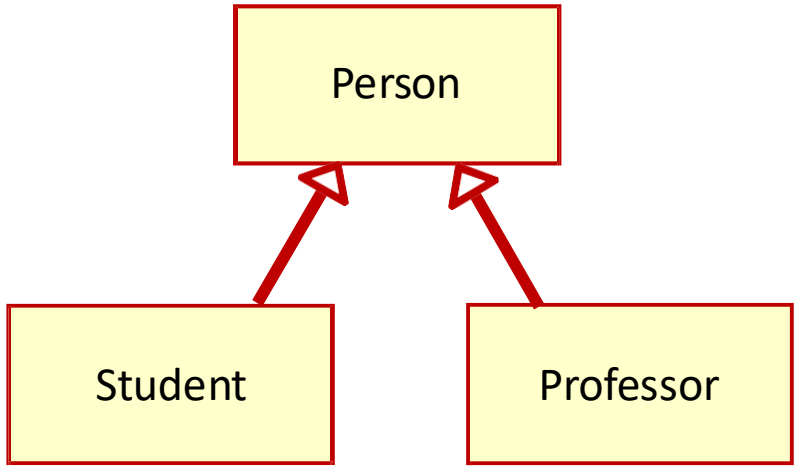


Σχέση: Γενίκευση (Generalization)



- Υποδηλώνει κληρονομικότητα
- Η κλάση παιδί κληρονομεί όλα τα χαρακτηριστικά, λειτουργίες και σχέσεις της κλάσης πατέρας
- Σχέση “is-a”
- Δεν μπορεί να χρησιμοποιηθεί με πολλαπλότητα

- Παράδειγμα:
  - Ο φοιτητής είναι άνθρωπος
  - Ο καθηγητής είναι άνθρωπος

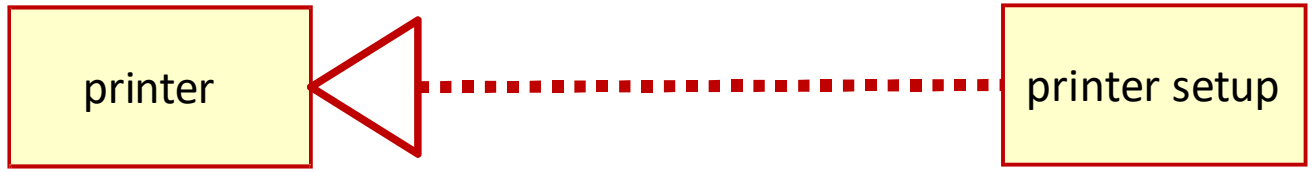


Σχέση: Πραγματοποίηση (Realization)

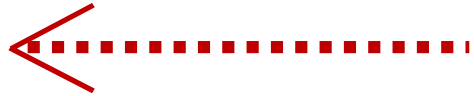


- Χρησιμοποιείται όταν μία κλάση πρέπει να υλοποιήσει ένα interface
- Υποδηλώνει ότι η λειτουργικότητα που καθορίζεται από μία κλάση πρέπει να υλοποιηθεί από κάποια άλλη

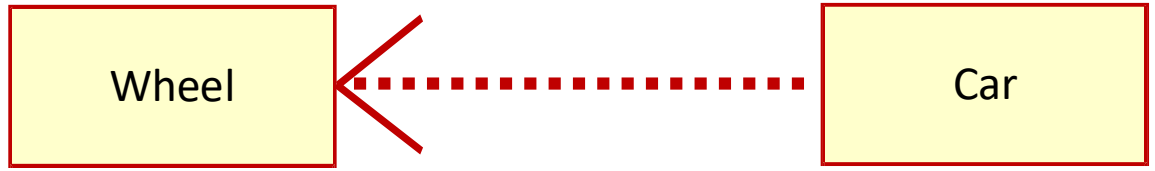
- Παράδειγμα:
  - Το printer πρέπει να υλοποιήσει τις επιλογές (printing preferences) που έχουν καθοριστεί από τον χρήστη



Σχέση: Εξάρτηση (Dependency)



- Υποδηλώνει εξάρτηση μεταξύ κλάσεων για κάποιο χρονικό διάστημα
- Πιο αδύνατη σχέση
- Μπορεί να προκληθεί όταν μία κλάση χρησιμοποιεί την άλλη μέσα από:
  - Τοπική μεταβλητή
  - Παράμετρο
  - Τιμή επιστροφής
- Παράδειγμα:
  - Το αυτοκίνητο εξαρτάται από τους τροχούς



Σχέση: Πολλαπλότητα (Multiplicity)

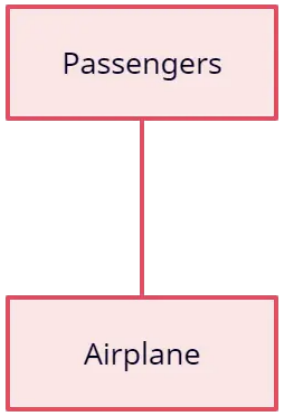
- Κάποιες συσχετίσεις μπορεί να έχουν ποσοτικούς περιορισμούς
- Η πολλαπλότητα δηλώνει σε πόσα αντικείμενα μπορεί να αναφέρεται η πηγή του αντικειμένου

- Παράδειγμα:
  - Ένας καθηγητής έχει τουλάχιστον ένα μάθημα Ένα μάθημα έχει ακριβώς ένα καθηγητή

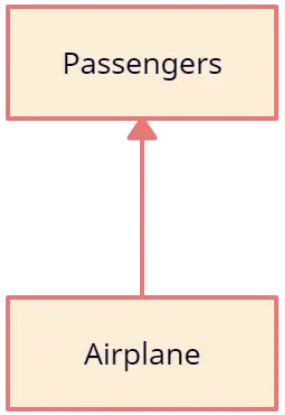
Συμβολισμός	Επεξήγηση
0..1	Zero or one
1	One only
0..*	Zero or more
*	Zero or more
1..*	One or more
3	Three only
0..5	Zero to Five
5..15	Five to Fifteen



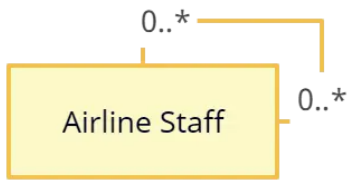
Παραδείγματα: Σχέσεις



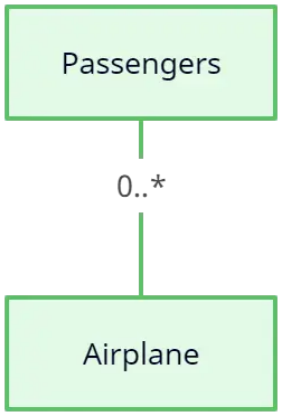
Association



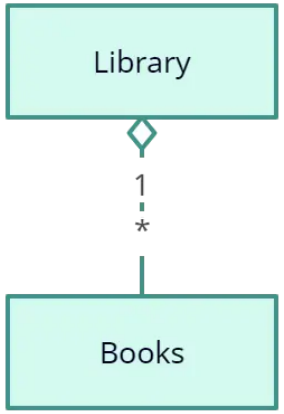
Directed Association



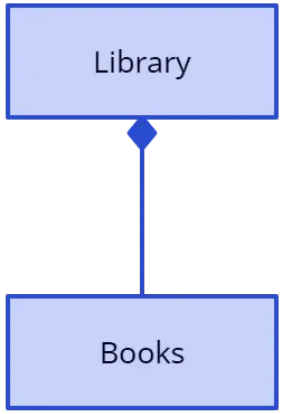
Reflexive Association



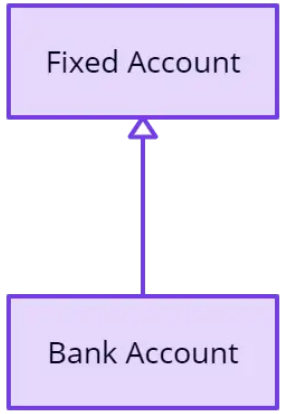
Multiplicity



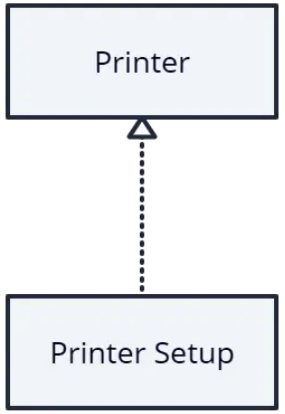
Aggregation



Composition

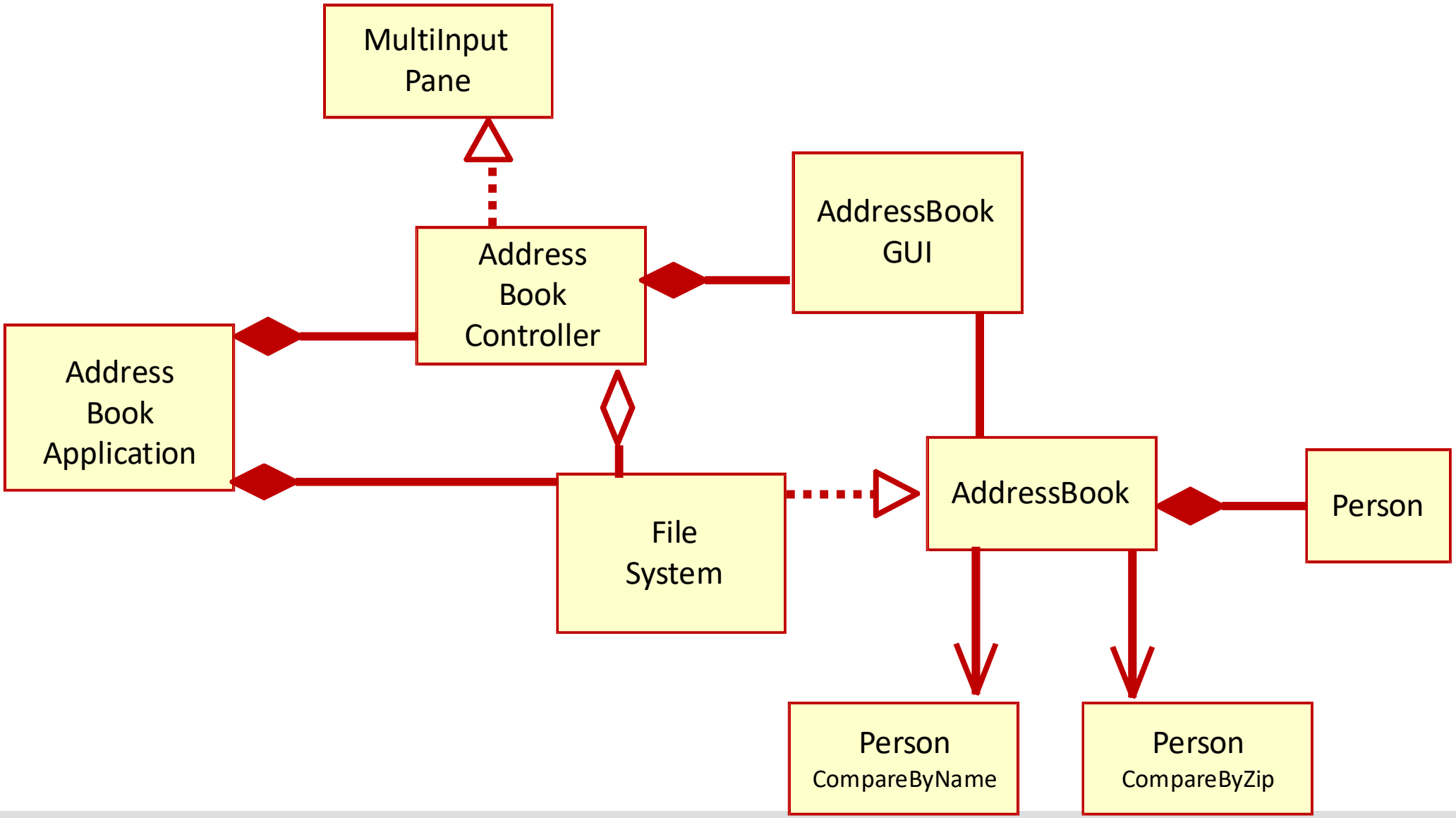


Inheritance



Realization

Βιβλίο Διευθύνσεων: Class Diagram



Παράδειγμα: Βιβλίο Διευθύνσεων
Φάση 3:Υλοποίηση

Φάση 3: Υλοποίηση

- Φάση της μετατροπής του αρχικού σχεδιασμού σε πρόγραμμα προς μετάφραση και εκτέλεση

3Α. Αναγνώριση Τμήματος Πυρήνα

- **Πυρήνας**: Στόχος εδώ είναι να εντοπίσετε το τμήμα που αποτελεί τον πυρήνα του προγράμματός σας, από τον οποίο θα προκύψει το τελικό σύστημα
- Ο πυρήνας μπορεί να μην περιέχει τα πάντα, είναι όμως η βάση για το χτίσιμο των λειτουργιών του συστήματός σας.

Φάση 3: Υλοποίηση (συν.)

3Β. Πρόσθεση Λειτουργιών

- Μετά το κτίσιμο του πυρήνα, γίνεται η υλοποίηση των λειτουργιών του συστήματος
- Κάθε λειτουργία αντιπροσωπεύεται στη φάση τού σχεδιασμού από μια αντίστοιχη περίπτωση χρήσης (use case)
- Η υλοποίηση των διαφορετικών περιπτώσεων χρήσης γίνεται διαδοχικά – σε διαφορετικές επαναλήψεις – και μέχρι να ολοκληρωθεί η υλοποίηση όλων των συστατικών του συστήματος

Διαγράμματα Κλάσης (class diagrams)

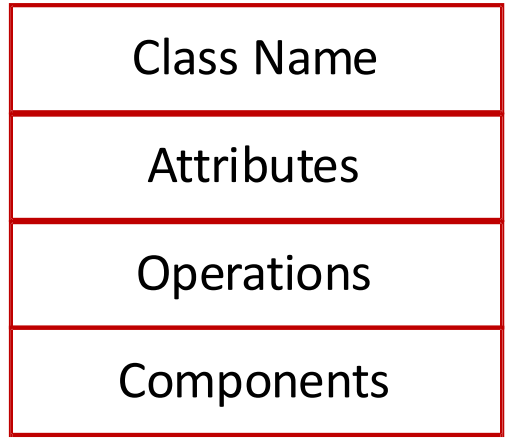
- Ένα από τα κυριότερα εργαλεία αντικειμενοστρεφούς μοντελοποίησης
- Περιγράφει τη δομή του συστήματος παρουσιάζοντας:
  - τις κλάσεις του συστήματος
  - τις σχέσεις μεταξύ κλάσεων

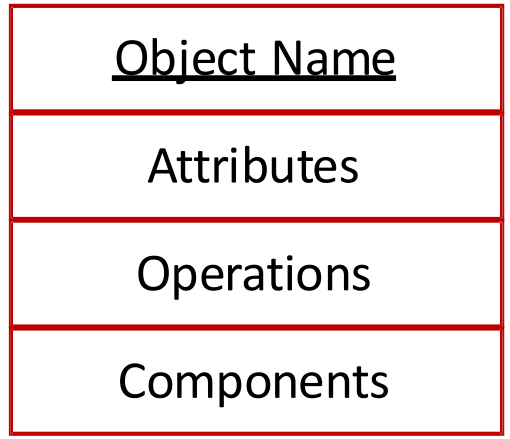
- Δύο είδη διαγραμμάτων:
  - **Γενικό εννοιολογικό μοντέλο (ΓΜ)**: χρησιμοποιείται στην ανάλυση και σχεδίαση
  - **Αναλυτικό μοντέλο (ΑΜ)**: χρησιμοποιείται για μετάφραση των μοντέλων σε πραγματικό κώδικα

Διαγράμματα Κλάσης (class diagrams) (συν.)

Αναλυτικό μοντέλο

- Διάγραμμα Κλάσης
  Κάθε κλάση χωρίζεται σε 4 μέρη
  1. Το όνομα της κλάσης (class name)
  2. Τα χαρακτηριστικά (attributes) της κλάσης
  3. Τις λειτουργίες (operations) της κλάσης
  4. (Προαιρετικό) Επιπρόσθετα συστατικά (additional components) της κλάσης
- Διάγραμμα αντικειμένου
  - Όμοιο με Διάγραμμα Κλάσης
  - Όνομα αντικειμένου αντί Όνομα Κλάσης
  - Όνομα αντικειμένου υπογραμμισμένο





Προσδιοριστές για Διαγράμματα Κλάσης

- Προσδιοριστές Ορατότητας (Visibility Modifiers)
  - **(+) public**: Η πρόσβαση είναι ανοικτή σε όλους!
  - **(-) private**: Η πρόσβαση είναι περιορισμένη μόνο στην κλάση
  - **(~) package (friendly)**: Η πρόσβαση είναι περιορισμένη σε κλάσεις που βρίσκονται στο ίδιο πακέτο με την κλάση
  - **(#) protected**: Όπως το default αλλά και σε κλάσεις που κληρονομούν από την ίδια κλάση (εντός πακέτου ή όχι)

Προσδιοριστές για Διαγράμματα Κλάσης

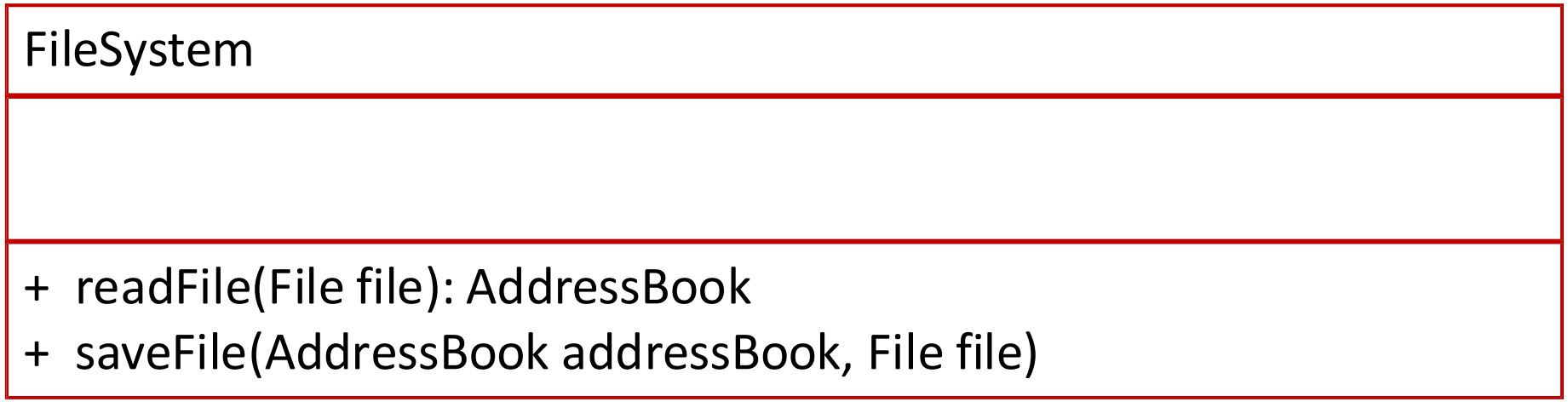
- Άλλοι Προσδιοριστές
  - **(underline) static**: Τα δεδομένα, μέθοδοι ισχύουν για όλα τα αντικείμενα της κλάσης
  - **(/) derived**: Τα δεδομένα, μέθοδοι κληρονομούνται από άλλες κλάσεις
  - **(italic) abstract**: Οι μέθοδοι πρέπει να υλοποιηθούν

Συμβολισμός	Επεξήγηση
+	Public
-	Private
#	Protected
~	Package
/	Derived
<underline>	Static
<italic>	Abstract

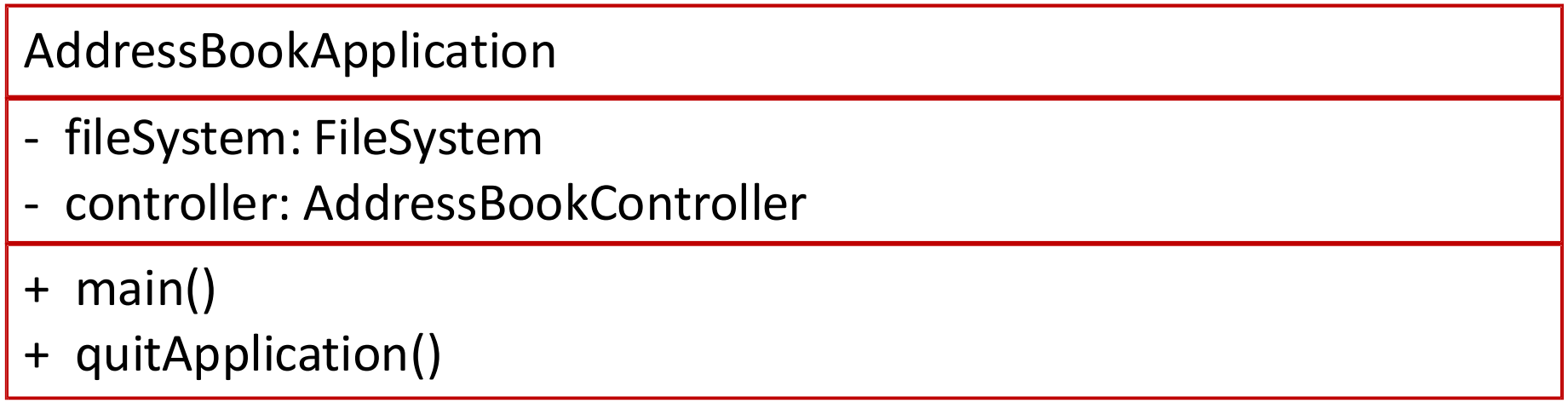
Βιβλίο Διευθύνσεων: Class Diagrams (αναλυτικά)

Person
- firstName: String - lastName: String - address: String - city: String - state: String - zip: String
+ Person(String firstName, String lastName, String address, String city, String state, String zip) + getFirstName(): String + getLastName(): String + getAddress(): String + getCity(): String + getState(): String + getZip(): String

Βιβλίο Διευθύνσεων: Class Diagrams (αναλυτικά)



Βιβλίο Διευθύνσεων: Class Diagrams (αναλυτικά)



Βιβλίο Διευθύνσεων: Class Diagrams (αναλυτικά)

AddressBook
- collection: Person [ ] or ArrayList - count: int - file: File - changedSinceLastSave: boolean
+ AddressBook() + getNumberOfPersons(): int + addPerson(String firstName, String lastName, …) + getFullNameOfPerson(int index): String + updatePerson(int index, String address, String city, String state, …) + removePerson(int index) + sortByName() + sortByZip() + printAll() + sortByName() + getFile(): File + getTitle(): String …

89

Παράδειγμα: Βιβλίο Διευθύνσεων
Φάση 4: Συντήρηση

Φάση 4: Συντήρηση

- Η φάση της συντήρησης ή εξέλιξης του κώδικα:
  - Διόρθωση σφαλμάτων
  - Αλλαγές στον κώδικα με βάση την εμπειρία χρήσης του
  - Πρόσθεση νέων λειτουργιών
  - Καλύτερη ικανοποίηση των απαιτήσεων

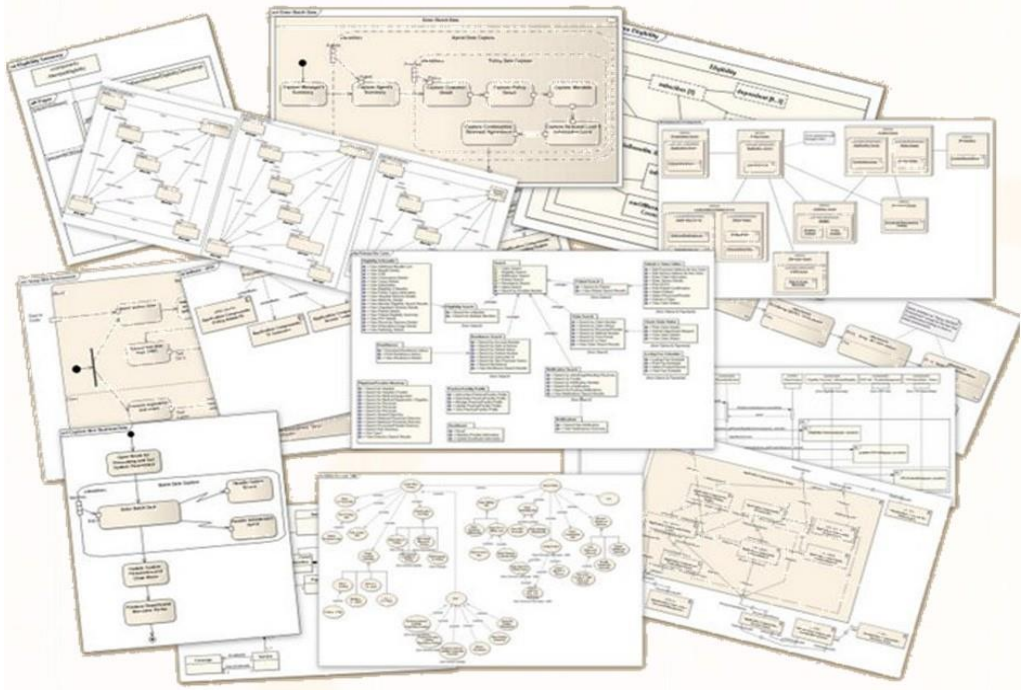
Βιβλίο Διευθύνσεων: Συντήρηση/Εξέλιξη

- Το **use case** **Print Entries** τυπώνει τα ονόματα στο *System.out*
  - Διαφορετικά θα μπορούσε να γράφει σε αρχείο
- Το πρόγραμμα μπορεί να διαχειρίζεται πολλαπλά βιβλία διευθύνσεων. Αυτό συμπεριλαμβάνει τις εξής αλλαγές:
  - Τα **use case** **Create New Address Book** και **Open Existing Address Book** δεν θα κλείνουν το τρέχον address book. Αντίθετα θα δημιουργούν ένα καινούριο αντίγραφο του GUI και θα το διαχειρίζονται ξεχωριστά.
  - Θα δημιουργηθεί ένα καινούριο **use case**, το **Close Address Book**, το οποίο θα εκτελείται όταν κλείνει ένα address book αντί του **use case** **Quit Program**

UML: Unified Modelling Language

UML: Unified Modelling Language

- Τυποποιημένη γλώσσα μοντελοποίησης γενικού σκοπού στο πεδίου της αντικειμενοστρεφή ανάπτυξης λογισμικού (object- oriented software engineering)
- Δημιουργήθηκε από το Object Management Group το 1997
- Η UML περιλαμβάνει ένα σύνολο από διαγραμματικές τεχνικές αναπαράστασης για να δημιουργήσει visual models



Κατηγορίες Διαγραμμάτων UML

- Δύο βασικές κατηγορίες διαγραμμάτων:
  - **Διαγράμματα Δομής (structure diagrams):** Δίνουν έμφαση στο τι πρέπει να είναι παρόν στο σύστημα που μοντελοποιείται, δηλ., την **αρχιτεκτονική** του
  - **Διαγράμματα Συμπεριφοράς (behavior diagrams):** Δίνουν έμφαση στο τι πρέπει να γίνει στο σύστημα που μοντελοποιείται, δηλ., την **λειτουργικότητά** του

Διαγράμματα Δομής (structure diagrams)

- **Class diagram:** περιγράφει τη δομή του συστήματος με κλάσεις, χαρακτηριστικά και σχέσεις
- **Component diagram:** περιγράφει πως ένα σύστημα είναι διαχωρισμένο σε συστατικά (components) και τις εξαρτήσεις μεταξύ αυτών
- **Composite structure diagram:** περιγράφει την εσωτερική δομή μίας κλάσης και τις συνεργασίες που επιτυγχάνει
- **Deployment diagram:** περιγράφει το υλικό (hardware) που χρησιμοποιεί το σύστημα και τα περιβάλλοντα εκτέλεσης που αναπτύσσονται σε αυτό.
- **Object diagram:** περιγράφει μία πλήρης ή μερική όψη της δομής του συστήματος σε κάποιο συγκεκριμένο χρόνο
- **Package diagram:** περιγράφει πως το σύστημα είναι διαχωρισμένο σε λογικές κατηγορίες και ποιες είναι οι εξαρτήσεις μεταξύ τους

Διαγράμματα Συμπεριφοράς (behavior diagrams)

- **Activity diagram:** περιγράφει ροές εργασίας (workflows) και ροές ελέγχου (flow of control) σε ένα σύστημα
- **State machine diagram:** περιγράφει καταστάσεις και μεταβάσεις του συστήματος
- **Use Case Diagram:** περιγράφει τη λειτουργικότητα του συστήματος με ρόλους/χρήστες και λειτουργίες
- **Communication diagram:** περιγράφει την επικοινωνία μεταξύ αντικειμένων με μηνύματα. Είναι συνδυασμός Class, Sequence, and Use Case Diagrams
- **Interaction overview diagram:** κάθε κόμβος είναι ένα communication diagram
- **Sequence diagram:** Παρόμοιο με communication diagram με τη διαφορά ότι κάθε αντικείμενο παρουσιάζει και το χρόνο ζωής του
- **Timing diagrams:** περιγράφει χρονικούς περιορισμούς

UML: Τι έχουμε μάθει;

- Use Case diagram
- Class Analysis diagram
- State machine diagram
- Sequence diagram
- Class diagram
- Detailed Class diagram

UML Παραδείγματα

- Κατανόηση των UML διαγραμμάτων
- Κατανόηση συσχετίσεων
- Παραδείγματα
  - Κατοικίδια ζώα και Ιδιοκτήτες
  - Οργάνωση σκληρών δίσκων
  - Τραπεζικό σύστημα
  - Σύστημα κεντρικής θέρμανσης
  - Σύστημα εκτύπωσης

UML Example – Veterinary System

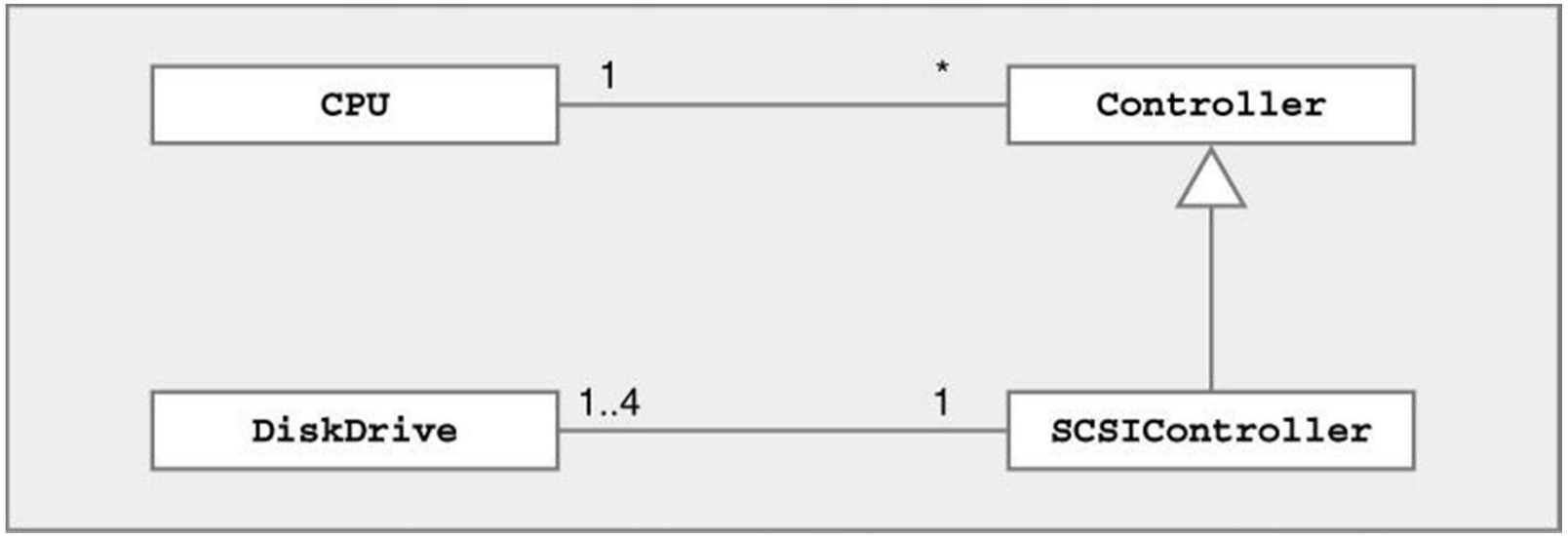
- Try to read & understand UML diagram



1 or more Pets associated with 1 PetOwner

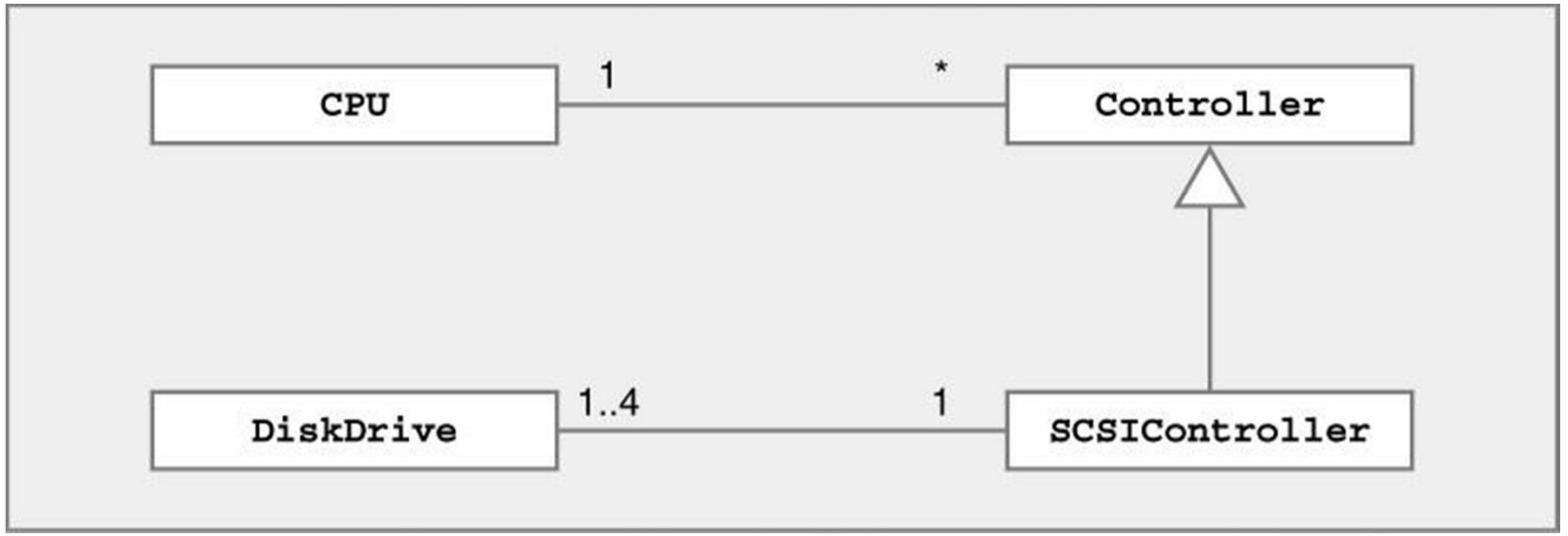
UML Example – Computer System

- Try to read & understand UML diagram



UML Example – Computer System

- Try to read & understand UML diagram

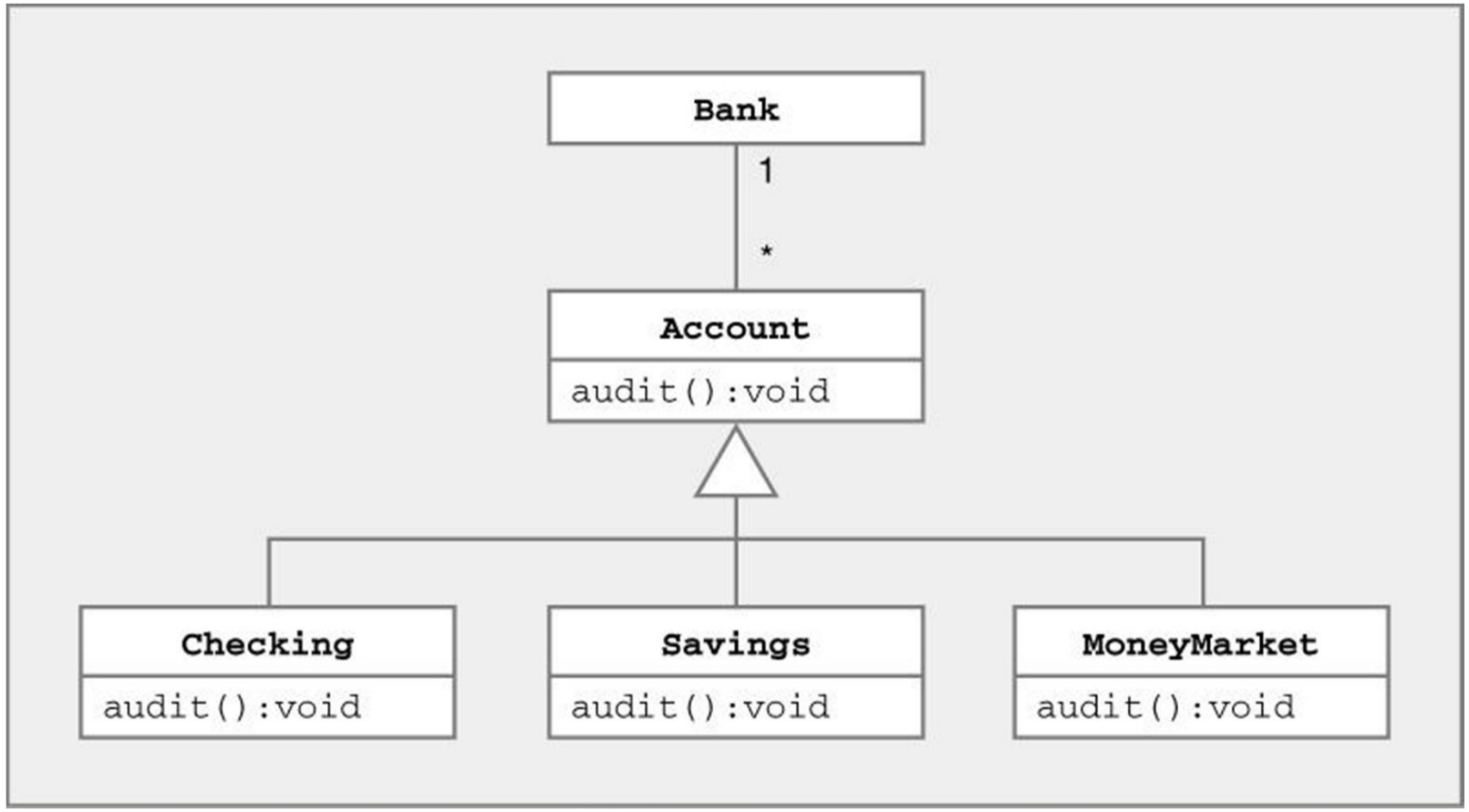


1 CPU associated with 0 or more Controllers

1 - 4 DiskDrives associated with 1 SCSIController

SCSIController is a (specialized) Controller

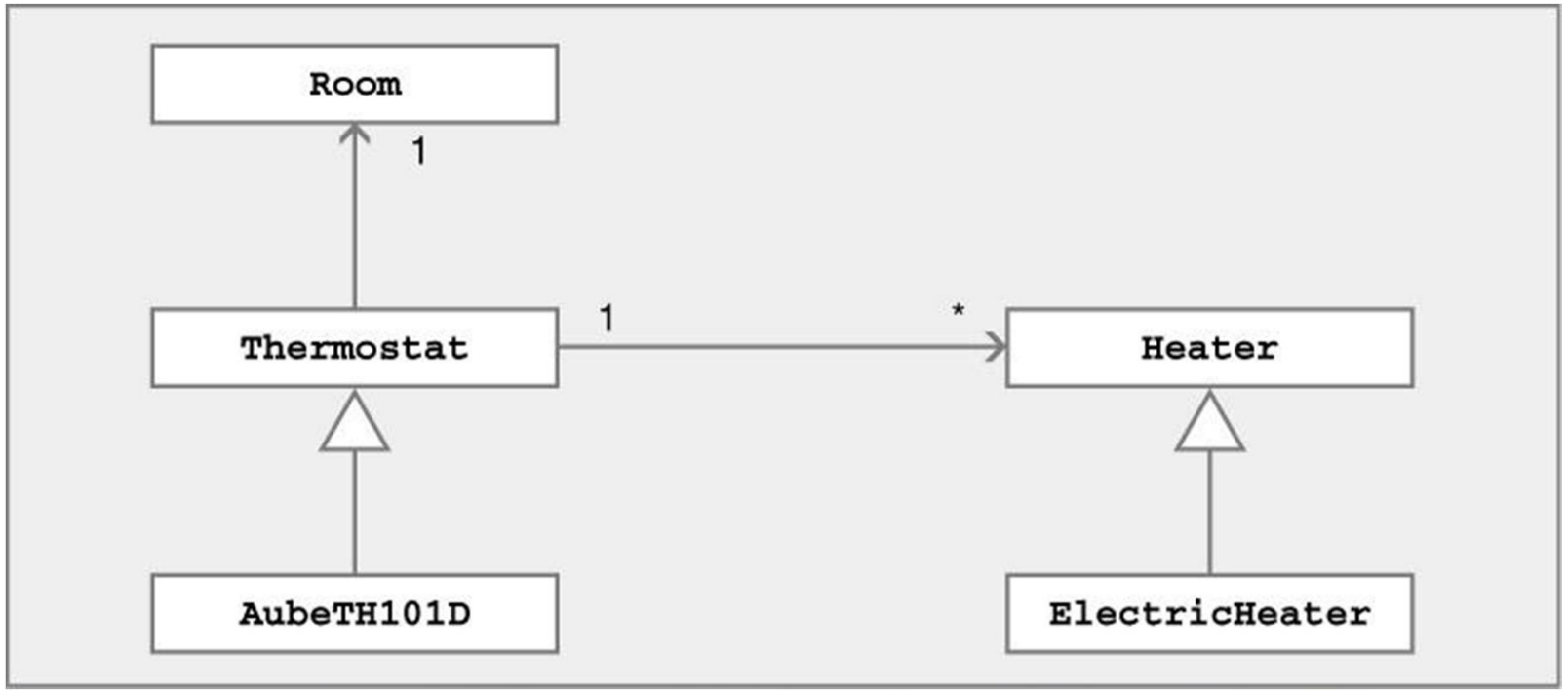
UML Example – Banking System



1 Bank associated with 0 or more Accounts

Checking, Savings, MoneyMarket are Accounts

UML Example – Home Heating System



Each Thermostat associated with 1 Room

Each Thermostat associated with 0 or more Heaters

ElectricHeater is a specialized Heater

AubeTH101D is a specialized Thermostat

UML Class Diagrams & Java

- Διαφορετική αναπαράσταση της ίδιας πληροφορίας
  - Name, state, behavior of class
  - Relationship(s) between classes

- Χρειάζεται εξάσκηση έτσι ώστε να μπορούμε να μετατρέπουμε το ένα στο άλλο και να αναπαριστούμε με ακρίβεια τα μοντέλα

UML & Java : Veterinary System

- UML



- Java

```
class Pet {
        PetOwner myOwner;        // 1 owner for each pet
}
class PetOwner {
        Pet[ ] myPets;      // multiple pets for each owner
}
```

UML & Java : Veterinary System

- Java

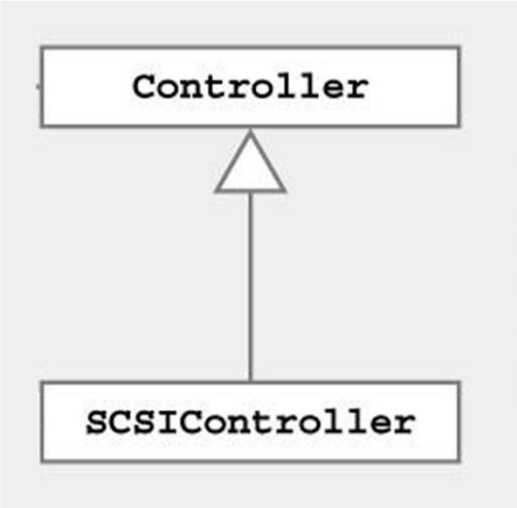
```
class Pet {
        PetOwner myOwner;        // 1 owner for each pet
}
class PetOwner {
        Pet[ ] myPets;      // multiple pets for each owner
}
```

- UML



UML & Java : Computer System

- UML



- Java

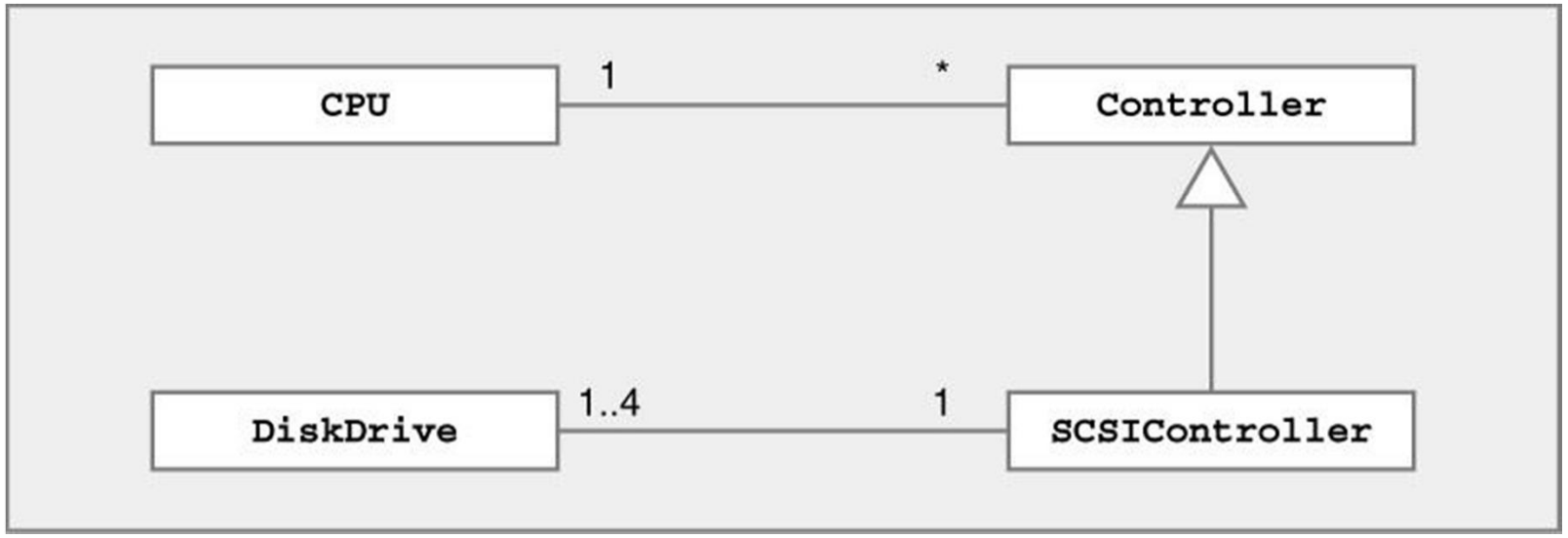
```
class Controller {
}
class SCSIController extends Controller {
}
```

κληρονόμος

Μητρική Κλάση

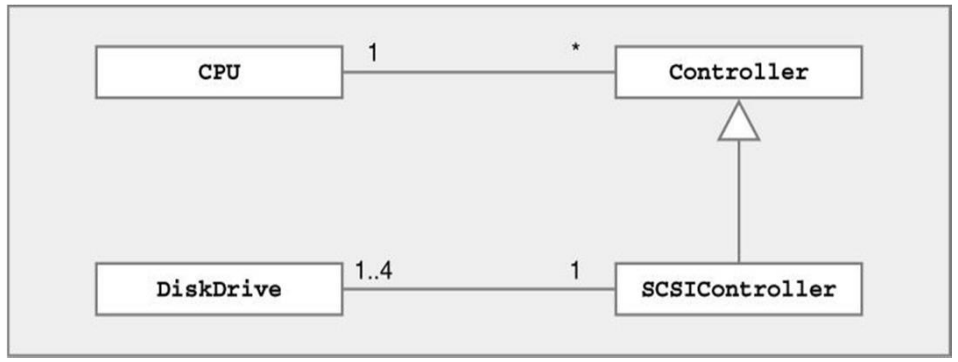
UML & Java : Computer System

- UML



UML & Java : Computer System

- Java

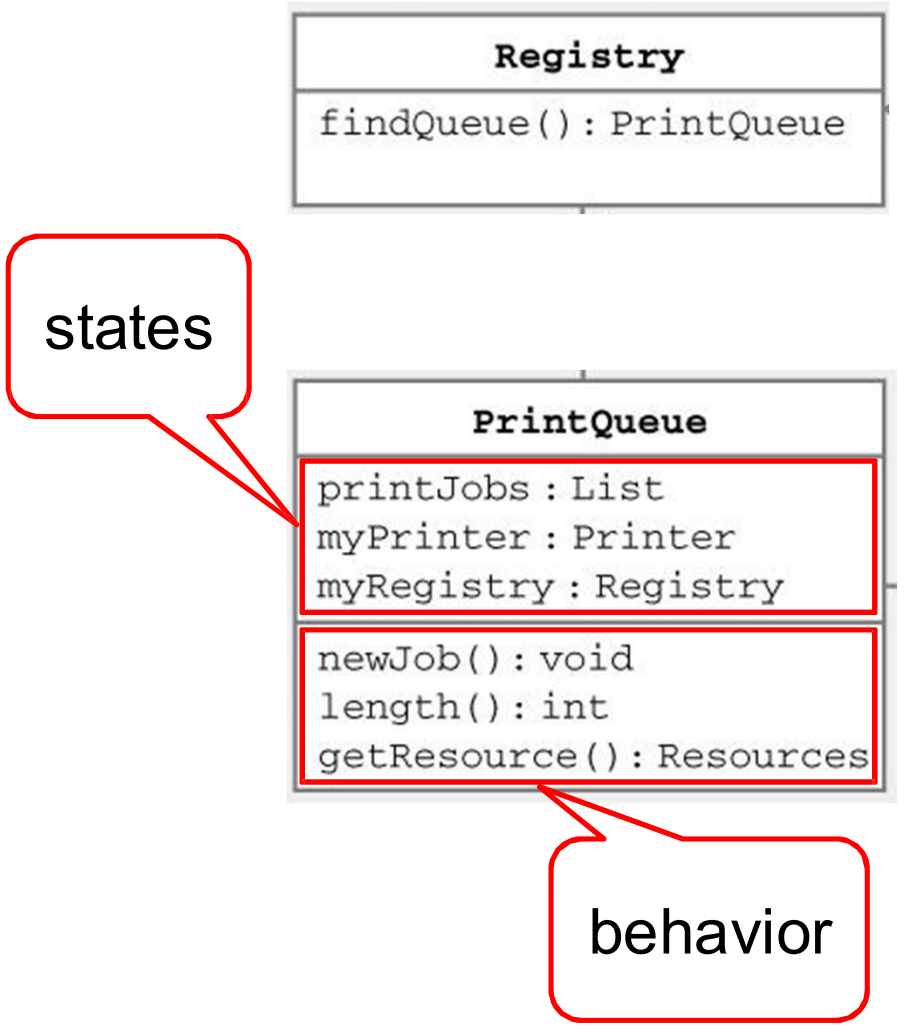



```
class CPU {
        Controller[] myCtlrs;
}
class Controller {
        CPU myCPU;
}
class SCSIController extends Controller {
        DiskDrive[ ] myDrives = new DiskDrive[4];
}
Class DiskDrive {
        SCSIController mySCSI;
}
```

Java & UML : Printing System

- Java

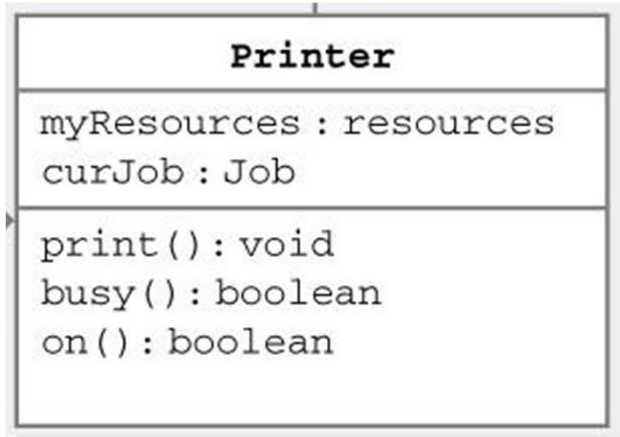
```
class Registry {
  PrintQueue findQueue();
}
class PrintQueue {
  List printJobs;
  Printer myPrinter;
  Registry myRegistry;
  void newJob();
  int length();
  Resources getResource();
}
```



Java & UML : Printing System

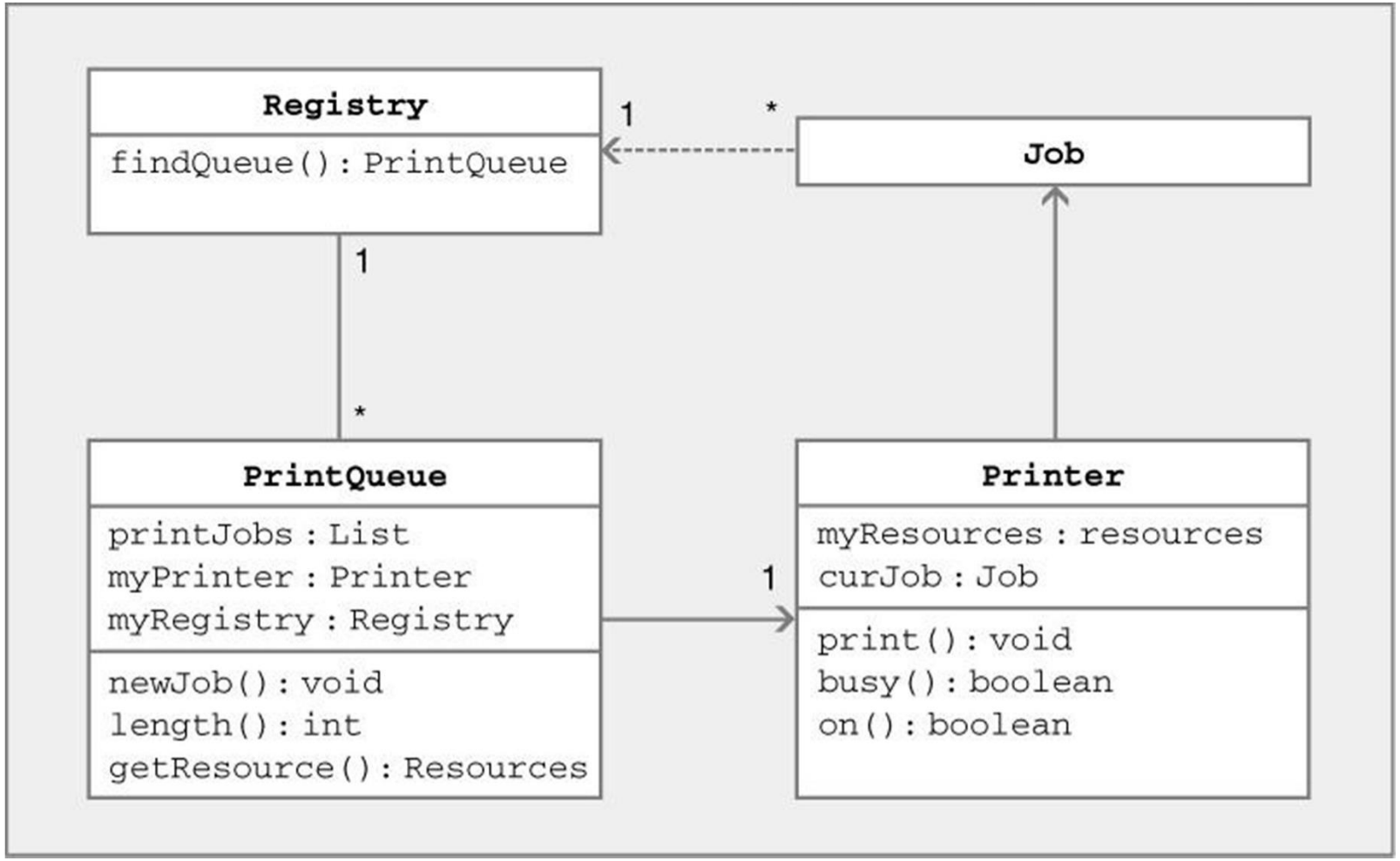
- Java

```
Class Printer {
   Resources myResources;
   Job curJob;
   void print();
   boolean busy();
   Boolean on();
}
Class Job {
   Job(Registry r) {
   ...
   }
}
```





Java & UML : Printing System



UML Σύνοψη

- Γλώσσα γραφικής μοντελοποίησης
- Γραφική απεικόνιση των σχεδιαγραμμάτων του συστήματος
- Εστιάσαμε περισσότερο στα διαγράμματα κλάσεων
  - Περιεχόμενα της κλάσης
  - Συσχετίσεις μεταξύ των κλάσεων
- Πρέπει να είστε σε θέση να:
  - Γράφετε τον κώδικα Java από το UML διάγραμμα
  - Σχεδιάζετε τα UML διαγράμματα από τον κώδικα Java